

Multi-Scale Urban Traffic Forecasting Using Graph-Based Deep Learning

MASTER THESIS

for the attainment of the Master's Degree

Master of Science MSc

at the Faculty of Digital and Analytical Sciences
of the Paris-Lodron-University of Salzburg

submitted by

TIMM HOPP

Supervisor: Prof. Dr. Johannes Scholz
Department of Geoinformatics

Salzburg, March, 2026

Declaration of Authorship

I hereby declare that I have written the present Master's thesis independently and without the use of sources or aids other than those indicated. All passages that are taken verbatim or in substance from published or unpublished sources have been clearly identified as such.

Generative AI tools were used exclusively for code polishing and for translating text; all scientific content, analysis, and conclusions are my own.

This thesis has not been submitted, in whole or in part, to any other examination authority and has not yet been published. •

I am aware that the submission of a false declaration may have legal consequences.

Salzburg, 12.03.2026



A handwritten signature in black ink, appearing to read 'Heg', is written over a horizontal line.

Abstract

Accurate forecasting of urban traffic is essential for mobility management, urban planning, and intelligent transportation systems but the strong spatial and temporal dependencies and complexities make it a difficult task. This thesis proposes a deep learning method that models traffic as a dynamic graph and models short-term patterns and long-term trends simultaneously. The architecture combines a Graph Attention Network with a two-fold temporal learning approach. Parallel Gated Recursive Units are used to learn rapid traffic changes on a short time frame, while a Long Short-Term Memory Recurrent Neural Network captures evolving daily patterns. The model is evaluated on real-world taxi data from 2024 in New York City. Data was aggregated as origin-destination matrices and represented as a dynamic graph. Experiments are conducted for next-step predictions and long-term forecasting across horizons between six hours and seven days. The model achieves consistently accurate predictions, with single-step MAE between 2.23 and 2.69 and an RMSE of 3.58 to 4.90 and cosine similarities of 0.88 to 0.98. Longer forecasts into the future yield similar results with long-term MAE of 2.53 to 2.77, RMSE of 4.31 to 4.76, and consistent cosine similarities around 0.88. A systemic bias towards underprediction is found for short-term forecasts but shown as correctable through simple linear regression based on residuals. The results demonstrate that the model is capable of successfully forecasting traffic at different scales, preserving spatial structures and temporal dependencies.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Literature Review	3
1.2.1	Classical Approaches in Traffic Modeling	3
1.2.2	Deep Learning for Graph-Based Traffic Forecasting	4
1.2.3	Temporal Modeling of Graph-Based Traffic	4
1.3	Open Challenges	5
1.4	Research Question	5
2	Methods	7
2.1	Graph	7
2.2	Model structure	7
2.3	Graph Attention Network	9
2.4	Temporal Learning	11
2.4.1	Long short-term memory (LSTM)	11
2.4.2	Gated Recursive Unit (GRU)	14
2.4.3	Temporal Attention	15
2.5	Decoder	16
2.6	Loss Function	17
3	Experiments	19
3.1	Setting	19

3.2	Data	19
3.3	Data Pre-Processing	20
3.4	Metrics	25
3.5	Training	27
4	Results	30
4.1	Single-Step Performance	30
4.2	Baseline Comparison	35
4.3	Post-hoc Correction	38
4.4	Long-Term Predictions	43
4.5	Attention Distribution	47
4.5.1	Spatial Attention	47
4.5.2	Temporal Attention	49
5	Discussion	53
5.1	Discussion of the Methods	53
5.2	Discussion of the Results	54
5.3	Future Work	56
6	Conclusion	58
	References	60
	Appendix	65

1 Introduction

1.1 Motivation

Accurate traffic forecasting is increasingly recognized as a key factor in improving sustainable urban mobility. Approximating future traffic, more efficient transport systems and reduced emission stemming from traffic congestion can directly help achieving the United Nations' Sustainability Goals (United Nations, 2015) in regards to 17 SDGs, including "Sustainable Cities and Communities" (SDG 11) and "Climate Action" (SDG 13). The United Nations (UN) identify Intelligent Transport Systems (ITS) as an important concept for the mitigation of climate change through a reduction of greenhouse gas emissions. Further, improved road safety stemming from ITS can avert road accidents and reduce traffic related injuries and deaths (United Nations, 2026).

Similar emphasize on upgrading current traffic systems is voiced by the European Union (EU). Within the "Sustainable and Smart Mobility Strategy" action plan, the EU outline steps that need to be taken to achieve the goals set by the European Green Deal (European Commission, 2019). By 2030, the European Commission aims to deploy automated mobility systems at large scale, and by 2050 a "Trans-European Transport Network" is to be operational for sustainable and smart transport. The use of innovative approaches utilizing Artificial Intelligence (AI) is named as a key goal to enable smart and eco-friendly traffic throughout the European Union (European Comission, 2020). In a 2024 report, the importance of ITS is highlighted again. First developed and deployed smart solutions showed improvements in traffic management reduced congestion by integration of large amounts of data in mobility services (European Commission. Directorate General for Mobility and Transport., 2024).

Additional importance is brought to the problem by the negative impact of traffic on the economy. A EU special report found that congestion leads to an estimated cost of 270 billion Euros per year in the EU alone, thus making it on of the largest burden on local economies. At the same time, free-flow traffic leads to up to 30% increased productivity of workers when compared to congested regions. As the majority of Europeans live in urban areas, the impact on local economies is high (European Court of Auditors, 2020). Research conducted at the European Institute for Innovation & Technology (EIT) additionally found great economic benefits in implementing urban traffic-related measurements envisioned by the European Green Deal. Across all cities of different sizes investigated throughout Europe, an increase of revenue of up to €700 million from implementation of innovative services and intelligent demand management each was identified (Borgato et al., 2021). Homem De Gouveia and Wojewódzka-Król (2025) identify effective and functioning urban traffic systems as vital for

quality of life, economic competitiveness, and beneficial working conditions. Mobility has a direct impact on employment and employee happiness, resulting in economic growth and an overall positive economical impact on all scales.

With advances in ITS and the introduction of smart-city services, accurate traffic forecasting is steadily gaining importance to enhance traffic planning and management (Zhao et al., 2020, Sun et al., 2023) or assist in route planning (Zhang et al., 2023). Urban traffic, however, introduces difficulties to forecasting tasks due to its high complexity and non-linearity (Kong et al., 2020). Additionally, traffic is inherently spatially and temporally dependent due to the road network topology and movement through time and space (Zhao et al., 2020). These issues motivate to model traffic forecasting as a graph problem (Zheng et al., 2020) and spatial relations as non-euclidean graph structures (Han et al., 2022). The road network serves as the underlying graph and changes in traffic can be modeled as node or edge signals varying over time (Zheng et al., 2020). To overcome these problems, a variety of graph-specific deep learning approaches have been proposed in recent years (Wu et al., 2021, M. Li and Zhu, 2021, Sun et al., 2023).

By leveraging these advanced approaches, long-term forecasts could help creating smarter and more adaptive urban traffic systems. Identifying patterns in the evolution of traffic over extended periods of time allows to identify structural patterns rather than relying on short-term change. The integration of large amounts of data allows to explore how different factors interact and impact and traffic over time. Long-term predictions can also play a crucial role in mobility planning and policy assessment. Cities can use forecasts to evaluate the potential impact of changes to the road network or regulations on mobility behavior using data-driven approaches that reflect patterns under different scenarios. This is especially important, as the challenges and goals mentioned above remain in conflict with consistent individual, car-centered mobility in urban areas. Additionally, long-term predictions allow to identify current shortcomings in traffic systems by identifying seasonal trends or recurring issues. These insights can help mitigating problems arising from weather events, public gatherings, or disruptions. Understanding patterns of car traffic can further help identifying new public transport options to further reduce emissions and create fair public transport in urban areas, while reducing risks stemming from traffic accidents. As cities adopt ITS, the ability to generate reliable forecasts becomes an integral part of sustainable and modern traffic management, enabling more efficient, eco-friendly, and economically viable transportation.

1.2 Literature Review

1.2.1 Classical Approaches in Traffic Modeling

Early work on traffic forecasting predominantly relied on statistical time-series models, with a strong focus on short-term prediction for operational traffic management. Chandra and Al-Deek (2009) emphasize the importance of short-term freeway speed and volume prediction in advanced traveler information systems and highlight limitations of traditional ARIMA models, which typically rely on autocorrelation at a single location and under-utilize upstream-downstream interactions. By modeling cross-correlations between stations, their vector autoregressive formulation captures spatial dependencies neglected in earlier univariate approaches. Extending this idea, Stathopoulos and Karlaftis (2003) propose a multivariate state-space model combining observation and state equations to describe the evolution of traffic conditions through autoregression, demonstrating sensitivity to changing traffic regimes. For urban arterials, Hamed et al. (1995) estimate ARIMA-based models that integrate the most recent forecast error with current observations to improve short-term volume prediction. To address nonlinearities and complex temporal structures, several authors introduced neural and hybrid models. X. Jiang and Adeli (2005) develop a dynamic wavelet neural network that leverages self-similar and fractal properties observed in traffic flow, using autocorrelation-based input selection and incorporating time-of-day and day-of-week effects for both long- and short-term forecasting. Similarly, Vlahogianni et al. (2005) apply a genetic algorithm to optimize neural network architectures, enabling modeling of spatial and temporal representations. Min and Wynter (2011) propose a vector spatio-temporal autoregressive model that integrates road network characteristics, traffic data, and incident information to generate multi-step predictions in real time. Comparative studies further highlight the trade-offs between statistical and learning-based approaches. Lippi et al. (2013) show that seasonal autoregressive models enhanced with Kalman filtering can outperform support vector models, although learning-based methods become competitive when equipped with radial basis functions. Castro-Neto et al. (2009) introduce an online support vector regression (OL-SVR) framework and demonstrate that Gaussian maximum likelihood performs well for recurring patterns, whereas OL-SVR is more robust under atypical traffic conditions. Earlier work by Okutani and Stephanedes (1984) already demonstrated the potential of Kalman filtering for dynamic traffic volume prediction, where forecasts are iteratively updated based on recent errors and smoothed over time. Overall, these classical approaches establish the foundations of spatial, temporal, and multivariate modeling in traffic forecasting, while also revealing key limitations that motivate the transition toward graph-based and deep learning architectures.

1.2.2 Deep Learning for Graph-Based Traffic Forecasting

A fundamental module to learn spatial relationships on graphs are semi-supervised learning Graph Convolutional Networks as proposed by Kipf and Welling (2017). GCNs have been widely applied to traffic forecasting tasks (Guo et al., 2019, Zhao et al., 2020, Lv et al., 2021, Zhang et al., 2021, W. Jiang and Luo, 2022). By combining a GCN with sequential learning, Y. Chen et al. (2023) are able to accurately predict traffic in fixed time intervals. However, the learning mechanism treats all nodes of the graph with the same importance, potentially missing additional spatial relations. To extract hidden information and local changes in traffic, Liu et al. (2024) proposed the use of multiple graphs in graph learning forecasting. Additionally, the use of the Graph Attention Network (GAT) instead of a GCN is focused on, as the GCN’s equal treatment of nodes potentially fails to uncover hidden relations. Instead, neighborhood nodes are weighted without knowledge of the full graph or complex calculations inherent to GCNs (Veličković et al., 2017). This further allows to generalize on the graph without consideration of structure (Kong et al., 2020). In combination with temporal pattern learning, Z. Li et al. (2024) identify spatio-temporal relations in the road network itself, improving forecasting capabilities and utilizing hidden information potentially missed in different architectures. Tang and Zeng (2021) further implement information derived from traffic data into the attention mechanism, allowing the model to learn on average travel time and volume transitions in addition to temporal changes and road network structure.

1.2.3 Temporal Modeling of Graph-Based Traffic

Temporal dependencies in traffic problems are typically modeled using Recurrent Neural Networks (RNN) variants of Long-Short Term Memory (LSTM; Sak et al., 2014) or Gated Recursive Unit (GRU; Chung et al., 2014). These modules help automatically capturing temporal relationships in traffic patterns along spatial evolution. Zhao et al. (2020) utilize a GRU alongside a GCN to capture spatio-temporal tendencies in urban traffic settings. Y. Chen et al. (2023) combine a GRU with GAT to capture temporal dynamics of spatial attentions. The LSTM is often used when long-term forecasts are desired due to its larger complexity and capability of learning dependencies across longer time spans. J. Chen et al. (2021) demonstrate the foundational capabilities of learning graph information over time by combining a GCN and a LSTM. By combining a GAT with an LSTM and additional contextual data, T. Zhu et al. (2024) predict traffic based on highly enriched data on different temporal horizons. W. Jiang et al. (2024) combine a GCN with an LSTM and outperform other graph neural networks in predicting mobility patterns.

The application of these temporal RNNs suffers from similar problems as GCNs for spatial learning. Traffic change does not necessarily depend on strict time series evolution, but shows complex spatial dependence imposed by network topology and time-varying dynamics. To weight past time steps for their importance, Guo et al. (2019) and Park et al. (2020) include temporal attention mechanisms in traffic prediction models.

1.3 Open Challenges

While a wide range of model architectures for future traffic estimation exist, models for long-term predictions remain limited. Approaches mentioned above forecast on short time spans that might not be practical in real-world deployment. Yu et al. (2017) predict on 15-, 30-, and 45-minute horizons. Other models were trained and tested for estimation of up to 60-minutes into the future (J. Zhu et al., 2020, Lu et al., 2020, Zhang et al., 2023, T. Zhu et al., 2024). As a result, most models are tested on scenarios that would appear unfit for real-world deployment. While predicting traffic into the next hour has its use cases, urban or transport planning benefit from longer forecasts. Extending prediction horizons introduces additional complexity into model architectures but could potentially bridge the gap towards fulfilling requirements for planning tasks.

Recent studies emphasize that long-term traffic forecasting remains a general challenge. Existing models struggle with distribution shifts, evolving traffic patterns, and long temporal gaps that characterize real-world conditions (Yin et al., 2024). Attioui and Lahby (2025) further highlight that the majority of research focuses on short-term horizons, leaving long-term prediction mostly underexplored. Additional challenges with machine learning methods arise as difficulties with modeling nonlinear long-range dependencies continue to be present and accumulating errors over extended horizons shape results. These issues are further found in broader surveys identifying long-horizon forecasting (Bielecki and Hachaj, 2026). Even recent models based on transformer architectures are shown to be faced with significant challenges in capturing complex spatio-temporal dependencies over long time spans (Duan et al., 2026). Only a small number of current models are explicitly built to address long-term forecasting, underscoring the novelty and difficulty of this research direction (Shen et al., 2025).

1.4 Research Question

The aim of this Master’s thesis is to develop a deep learning-based approach to capture long-term trends and short-term patterns in traffic simultaneously to allow for predictions

farther into the future. The model should learn from historical data and produce accurate and reliable predictions. A two-branch architecture is implemented, trained, and tested on real-world data. To evaluate the model’s performance, predictions are made on the next timestep, as well as on longer temporal horizons to examine its ability to generalize beyond immediate forecasts.

For evaluation, three key aspects are focused on in regards to traffic patterns and volumes prediction:

1. **Short-term performance:** How well does the model capture rapid, short-term changes in traffic?
2. **Long-term performance:** How accurately does the model learn and represent trends over longer time spans?
3. **Baseline comparison:** Does the model provide improved predictions compared to statistical baseline methods?

As urban traffic systems are inherently dynamic and show steady change across multiple time scales, it is important to create a model that enables reliable forecasting on short- and long-term traffic simultaneously. Short-term fluctuations, such as rush hours, weather events, or road incidents, directly impact traffic patterns on a local scale, while long-term trends driven by the distribution of population, areas of interest, commuting behaviors, or seasonal cycles, influence traffic year-round. It is thus essential for a traffic prediction model to capture multiple scales to enable informed and sustainable decision-making.

Short-term accuracy is important for traffic control in near real-time to steric signal timing adaptations, route guidance, or restrictions. Long-term accuracy, on the other hand, gains increasing importance for strategic planning. Reliable predictions allow to evaluate infrastructure projects and policies. Traditional models often struggle with extended horizons by accumulation of error or reliance on unrealistic assumptions. Even modern models based on deep learning lack capabilities to forecast on extended time periods. A model bridging the gap between different temporal horizons therefore addresses an important and necessary issue. Cities require tools that support real-time monitoring of traffic while also allowing to identify long-term patterns to find strategies fit for the future. Capturing multiple temporal scales therefore directly contributes to a wide range of applications and fundamental understanding of traffic.

2 Methods

2.1 Graph

Data used by the model is organized as a dynamic directed graph $G_t = (V, E_t, A)$ at timestep t , where V are the vertices or nodes of the graph, E_t the edges connecting two nodes, and A the adjacency representation of G_t . Unique designated zones within the area of interest are considered as vertices with fixed spatial positions. Edges are encoded as connections between two zones and represented through traffic connections, topological relationships, and distances independent of time. Temporal change is introduced by processing a series of G_t subsequently. To allow the graph to be processed by the model, the data is transformed into a matrix $OD_t \in \mathbb{R}^{N \times N}$ with $OD_t[i, j]$ as the number of trips between two nodes i, j in a timestep t . The resulting OD matrices are stacked as a tensor $T_{in} = \{OD_1, OD_2, \dots, OD_k\} \in \mathbb{R}^{T \times N \times N}$, where T is the total number of timesteps and N the number of graph vertices.

2.2 Model structure

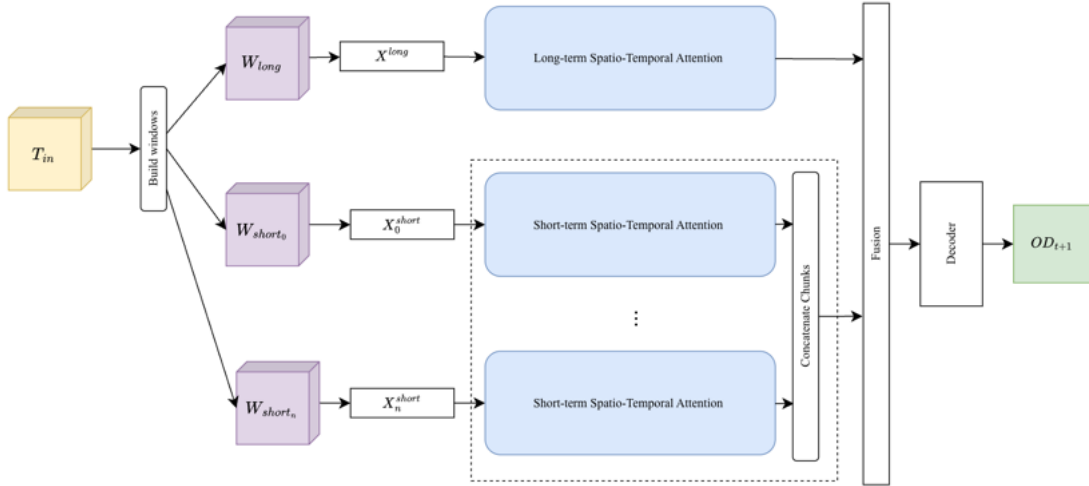


Figure 1: Two-Branch Model Structure

To allow the model to learn short- and long-term patterns simultaneously, a two-branch model was built (Fig.1). Daily traffic patterns are captured through a long-term branch that consists of a Graph Attention Network (GAT; Veličković et al., 2017), a Long Short-Term Memory unit (LSTM; Sak et al., 2014), and temporal attention mechanism (Cho et al., 2015) as shown in Figure 2a. Short-term changes are captured through multiple parallel short-term

modules, each consisting of a GAT, a Gated Recursive Unit (GRU; Chung et al., 2014), and temporal attention (Fig.2b).

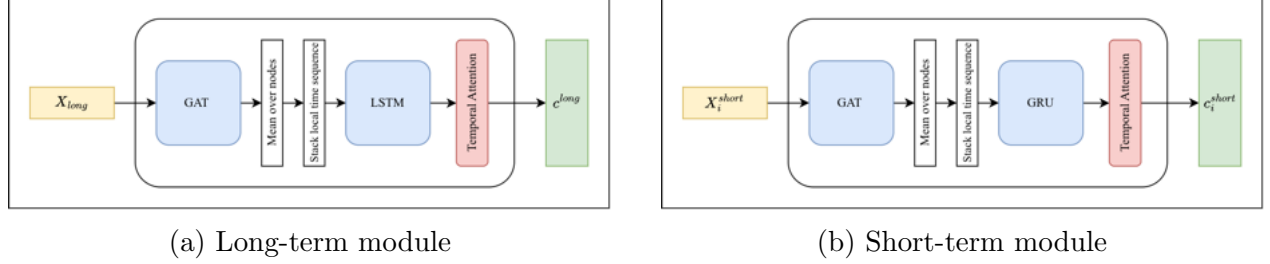


Figure 2: Structure of Modules for Different Temporal Scales

To ensure temporal consistency, the input matrices are processed as windows of defined sizes. The long-term branch receives a subset of T as W_{long} , representing the total time frame on which it will be trained. Short-term patterns are learned from equal parts of the same W_{long} , where

$$W_{short} = \frac{W_{long}}{K}$$

for K short-term spatio-temporal attention modules. The windows are read as node features $X_\tau \in \mathbb{R}^{N \times 1}$ derived from OD matrices.

After learning the data structure, the final hidden states of all short-term branches are concatenated and fused with the learned long-term attention. After processing by the decoder, a single OD matrix

$$\hat{Y} = OD_{t+1} \in \mathbb{R}^{N \times N}$$

at time $t + 1 = t_{W_{long}+1}$ is created by the model, representing the predicted traffic after the final matrix of W_{long} . Long-term predictions were made by predicting the matrix at fixed temporal distance d . Instead of estimating traffic at $t_{W_{long}+1}$, OD_{t+d} was produced. After processing the entire set of training matrices, negative binomial NLL loss is calculated by comparing all $\hat{Y}$ to corresponding ground-truth Y/Y_{t+d} and model weights and bias are adapted in backpropagation.

2.3 Graph Attention Network

To model the spatial structure of the underlying graph and learn influences of zones on each other, a Graph Attention Network (GAT) is used. Adaptive attention weights are assigned to graph nodes to allow the model to focus on the zones that contribute most strongly to local traffic dynamics. Relationships are not solely defined by static topology or physical distance but are learned from patterns within the data.

The GAT receives a sequence

$$X = X_{t-W+1}, X_{t-W+2}, \dots, X_t \in \mathbb{R}^{1 \times N}$$

of node features of a data window W and transposes it into an n -dimensional vector $X_\tau \in \mathbb{R}^{N \times 1}$, with τ representing the local time of the processed window. Each node is then embedded as

$$h_{i,\tau}^{(0)} = x_{i,\tau} \in \mathbb{R}.$$

In a first step, the GAT applies a linear projection of node embeddings into hidden space of dimension H by multiplication with linear weights $W \in \mathbb{R}^{H \times 1}$ as

$$h'_{i,\tau} = W h_{i,\tau}^{(0)} \in \mathbb{R}^H.$$

For each node i and each of its adjacent neighbors $j \in \mathcal{N}(i)$, pair representations are constructed as

$$z_{ij,\tau} = [h'_{i,\tau} \| h'_{j,\tau}] \in \mathbb{R}^{2H}.$$

Through multiplication with a transposed learnable attention vector $a^\top$ and nonlinearity by application of the Leaky Rectified Linear Unit (LeakyReLU) function, raw feature attention scores are calculated as

$$e_{ij,\tau}^{feat} = \text{LeakyReLU}(a^\top z_{ij,\tau}) \in \mathbb{R}.$$

Spatial distances are introduced into the GAT by calculating a distance-based influence term

$$e_{ij,\tau}^{dist} = -\log(D_{ij} + \epsilon) \cdot s \in \mathbb{R},$$

where ϵ is a very small ($1e-3$) positive number to avoid calculating the negative log of 0, and $s \in \mathbb{R}$ is a learnable scalar describing the influence of distance on node neighborhood attention scores. This ensures assigning higher influence to geographically close regions while decreasing attention with distance. The combined spatial attention score is then calculated as

$$e_{ij,\tau} = e_{ij,\tau}^{feat} + e_{ij,\tau}^{dist} \in \mathbb{R}.$$

To skip attention scores of non-existent edges in the data, the global static adjacency matrix A is used as a mask to set non-existent edges to 0:

$$\tilde{e}_{ij,\tau} = \begin{cases} e_{ij,\tau}, & A_{ij} = 1, \\ c_{neg}, & A_{ij} = 0, \end{cases} \quad c_{neg} \ll 0$$

To make attention weights interpretable and stable throughout the GAT, Softmax normalization

$$\alpha_{ij,\tau} = \frac{\exp(\tilde{e}_{ij,\tau})}{\sum_{k \in \mathcal{N}(i)} \exp(\tilde{e}_{ik,\tau})}$$

is applied. Attention scores are now normalized and positive and form a probability distribution over the neighborhood of each node. This allows the model to weigh attentions based on their importance of interaction. Use of $\alpha_{ij,\tau}$ removes the dependence of attention on learned parameter, distance, and feature magnitude and gives overall relative differences between neighboring nodes. Irrelevant neighborhoods are pushed towards 0, whereas greater spatial importance receives larger $\alpha_{ij,\tau}$ values.

Neighbor information is then aggregated as

$$h_{i,\tau}^{agg} = \sum_{j=1}^N \alpha_{ij,\tau} h'_{j,\tau} \in \mathbb{R}^H,$$

into new hidden embeddings with $h'_{j,\tau} = Wh_{j,\tau}^{(0)}$ and $h_{j,\tau}^{(0)} = x_{j,\tau}$. All aggregate node embeddings are stacked into $\hat{h}_\tau \in \mathbb{R}^{N \times H}$. Finally, a LeakyReLU activation is applied element-wise to obtain the spatial embedding at τ :

$$H_\tau^{GAT} = \text{LeakyReLU}(\hat{h}_\tau) \in \mathbb{R}^{N \times H}.$$

The resulting H_τ^{GAT} is then forwarded to the temporal module. The implementation of the Graph Attention Network is provided in Appendix B.

2.4 Temporal Learning

In order for the model to learn on different temporal scales, two modules were created. A long-term attention network extracts traffic patterns over time on the entirety of W_{long} , calculating the mean over nodes for the GAT outputs, and processing the stacked time sequences in an LSTM. A spatio-temporal context vector is generated from the learned temporal encoding in a temporal attention unit. For short-term learning, the LSTM is substituted by a GRU. Input segments are further split into equal segments and fed into separate branches, each being trained on the same, repeating time frame of input data. By processing the GAT outputs in subsequent order, temporal learning is achieved from a combination of spatial and temporal attention mechanisms and traffic patterns are identified directly from extracted graph information. Using this two-fold architecture allows the model to learn long-term trends without losing detail in rapid changes.

2.4.1 Long short-term memory (LSTM)

To learn temporal change across the graph, the aggregated spatial embeddings $H_\tau^{GAT} \in \mathbb{R}^{N \times H}$ are read and, through mean pooling, transformed into a single vector per timestep:

$$m_\tau = \frac{1}{N} \sum_{i=1}^N H_{\tau,i}^{GAT}$$

This vector is then treated as sequences of input data $\mathcal{M} = (m_{t-W_{long}+1}, \dots, m_t) \in \mathbb{R}^{W_{long} \times H}$ with $m \in \mathbb{R}^H$. An LSTM is used to learn long-term spatial trends in the data.

For each sequence of input hidden states, the LSTM calculates:

$$i_\tau = \sigma(W_i m_\tau + U_i h_{\tau-1} + b_i)$$

$$f_\tau = \sigma(W_f m_\tau + U_f h_{\tau-1} + b_f)$$

$$o_\tau = \sigma(W_o m_\tau + U_o h_{\tau-1} + b_o)$$

$$\tilde{c}_\tau = \tanh(W_c m_\tau + U_c h_{\tau-1} + b_c)$$

$$c_\tau = f_\tau \odot c_{\tau-1} + i_\tau \odot \tilde{c}_\tau$$

$$h_\tau = o_\tau \odot \tanh(c_\tau)$$

where h_τ and $h_{\tau-1}$ are the hidden states at τ and $\tau - 1$, and c_τ the cell state at τ . i_τ , f_τ , $\tilde{c}_\tau$, and o_τ are the input, forget, candidate cell, and output gates of the LSTM with results between 0 and 1 and function as soft filters. σ denotes the application of the sigmoid function and $\odot$ of the element-wise product calculation. Fig.3 shows the structure of such an LSTM cell. The full model implementation is provided in Appendix A.

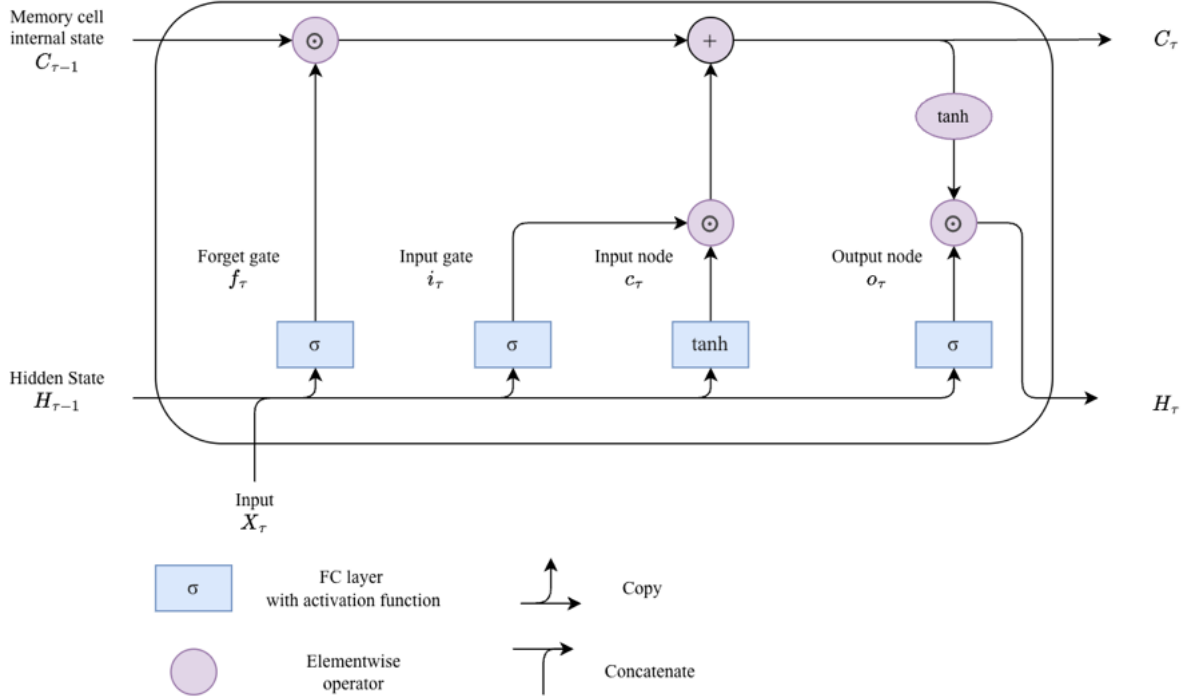


Figure 3: Gates and States of an LSTM Cell

The LSTM processes the pooled $\mathcal{M} \in \mathbb{R}^{W_{long} \times H}$ and adapts the gates' input weight matrices W_* and recurrent weight matrices U_* , allowing it to blend new inputs and memories from processed sequences through multiplication with the input vector. W_* are adapted to learn the influence of time on the gates and states by capturing changes in input over time, while U_* are modeling the impact of past hidden states in stored memory.

For each sequential step, the input gate i_τ determines how much input information is to be considered in updating weights. The forget gate f_τ controls removal of information from previous memory. The candidate cell gate $\tilde{c}_\tau$ generates potential new memory content based on learned weights. The new cell state c_τ is then formed by combining retained memory $f_\tau \odot c_{\tau-1}$ with newly proposed memory $\tilde{c}_\tau \odot i_\tau$, allowing the LSTM to integrate past context with current input and newly learned information by continuously updating internal memory. The output gate o_τ determines the contribution of updated cell states to new hidden states, calculated as

$$h_\tau = o_\tau \odot \tanh(c_\tau)$$

After processing of all time steps, the LSTM returns the final hidden state $h_T \in \mathbb{R}^H$.

2.4.2 Gated Recursive Unit (GRU)

Short-term changes in traffic are modeled using a GRU per time window W_{short} . As simplified variants of the LSTM, GRUs show lower model complexity while retaining capabilities to learn temporal patterns over shorter spans of time. Overfitting on short sequences becomes less likely by merging of the forget and input mechanisms of the LSTM into a single update gate and use of fewer parameters in computation. (Fig.4)

This reduced model complexity additionally allows for faster learning and adoption to rapid changes in observed patterns, making GRUs suitable for subsections of the data processed by the LSTM. Further, faster convergence allows for parallel training on different time segments if sub-patterns are distinguishable.

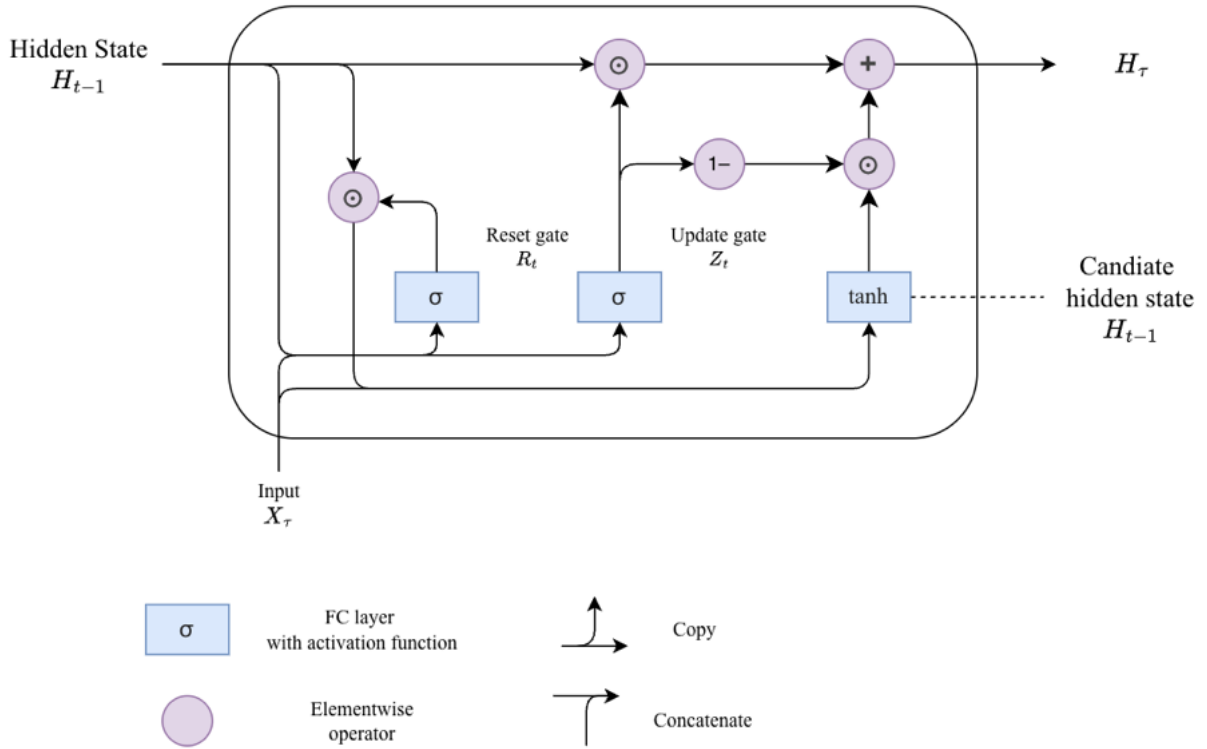


Figure 4: Gates and States of a GRU Cell

As the LSTM, the GRU receives spatial embeddings $H_\tau^{GAT} \in \mathbb{R}^{N \times H}$ from the GAT and applies mean pooling across the nodes to obtain $m_\tau \in \mathbb{R}^H$. At time τ the GRU calculates:

$$z_\tau = \sigma(W_z m_\tau + U_z h_{\tau-1} + b_z)$$

$$r_\tau = \sigma(W_r m_\tau + U_r h_{\tau-1} + b_r)$$

$$g_\tau = \tanh(W_g m_\tau + U_g (r_\tau \odot h_{\tau-1}) + b_g)$$

$$h_\tau = (1 - z_\tau) \odot h_{\tau-1} + z_\tau \odot g_\tau$$

The update gate z_τ decides how much data of the previous hidden state is kept or updated. Forgetting and removing past information from memory is handled by the reset gate r_τ . Analogous to the LSTM's candidate gate, g_τ calculates a possible new candidate hidden state from the current input and previous state. The candidate and hidden state are then blended into a new hidden state through h_τ .

After iterating over all τ , the GRU returns the final hidden state $h_t \in \mathbb{R}^H$.

2.4.3 Temporal Attention

After processing the data windows, both the LSTM and GRU return temporal sequences $\{h_\tau\}_\tau^T$ containing hidden states for $h_\tau \in \mathbb{R}^H$ with $\tau = t - W_{long/short} + 1, \dots, t$. To identify the most relevant information on temporal changes and reduce noise, temporal attention is calculated by assigning learnable weights to hidden states. Each h_τ is projected into attention space via

$$u_\tau = \tanh(W_h h_\tau + b_h) \in \mathbb{R}^H$$

where W_h are the learnable weights to identify importance. Bias is denoted as b_h . Hidden state relevance is then measured by calculating a temporal attention score by multiplying u_τ with a transposed learnable attention vector $v^\top$

$$e_\tau = v^\top u_\tau \in \mathbb{R}$$

Attention scores are then normalized by

$$\alpha_\tau = \frac{\exp(e_\tau)}{\sum_{k=1}^T \exp(e_k)} \in \mathbb{R}$$

and context vectors calculated and returned as

$$c = \sum_{\tau=1}^T \alpha_\tau h_\tau \in \mathbb{R}^H$$

If n -many short-term modules are used, the context vectors are additionally concatenated as $c^{short} = [c^{short_0}, \dots, c^{short_n}] \in \mathbb{R}^{nH}$.

Before passing the learned spatio-temporal patterns from hidden states to the decoder, long-term and short-term contexts are fused into a single vector $c^{fuse} = [c^{long}, c^{short}] \in \mathbb{R}^{(n+1)H}$. The temporal attention implementation is provided in Appendix C.

2.5 Decoder

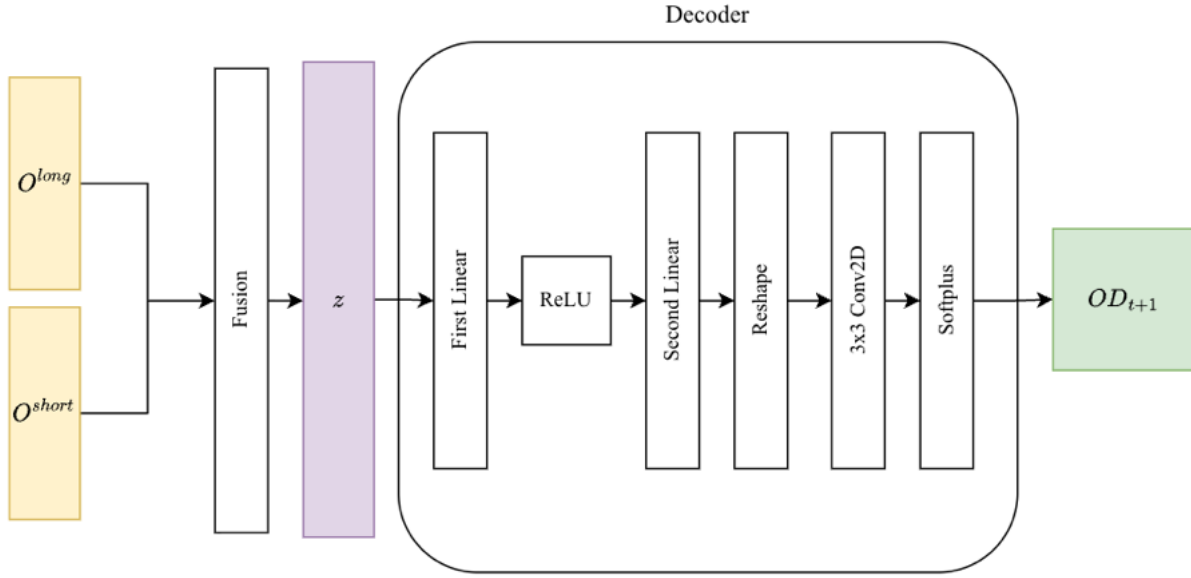


Figure 5: Diagram of Decoder Structure

In a final step, context scores are transformed into a single predicted OD matrix OD_{t+1} by decoding through multiple steps. (Fig.5) In a first linear projection, c^{fuse} is projected into $\mathbb{R}^{2H}$ by

$$z_1 = \mathbf{W}_1 c^{fuse} + b_1 \in \mathbb{R}^{2H}$$

with learnable weights $\mathbf{W}_1 \in \mathbb{R}^{(2H) \times (n+1)H}$ and bias $b_1 \in \mathbb{R}^{2H}$. Non-linearity is introduced by applying ReLU. The resulting $z_1^{\text{ReLU}} = \text{ReLU}(z_1) \in \mathbb{R}^{2H}$ is then passed into a second linear layer

$$z_2 = \mathbf{W}_2 z_1^{\text{ReLU}} + b_2$$

With learnable weights $\mathbf{W}_2$ of shape $\mathbb{R}^{(N \cdot N) \times (2H)}$ and bias $b_2 \in \mathbb{R}^{N \cdot N}$, the result is a flattened $N \times N$ -shaped OD matrix. By applying reshape, the resulting $Z_{OD} = \text{reshape}(z_2) \in \mathbb{R}^{N \times N}$ is transformed into the original matrix shape.

To capture local spatial interactions and apply smoothness, a 3x3 convolution kernel is applied on the derived matrix. This helps to capture additional local spatial correlations and smoother OD predictions. A 3x3 kernel is the minimal possible kernel size to capture spatial information on a local level. It was started with a padding factor of 1 to retain the structure of the matrix $N \times N$. Applying the kernel returns a prediction matrix $\hat{Y}^{conv} \in \mathbb{R}^{N \times N}$.

To enforce non-negative values, Softplus(x) = $\log(1 + e^x)$ was applied to $\hat{Y}^{conv}$. The resulting $\hat{Y} \in \mathbb{R}^{N \times N}$ is the predicted OD_{t+1} and final output of the model. The decoder implementation is provided in Appendix D.

2.6 Loss Function

To train the model for accurate OD traffic flow prediction, the Negative Binomial negative log-likelihood (NB-NLL) loss function (Salinas et al., 2020) was chosen. As OD traffic is represented as non-negative integer counts and naturally show strong levels of overdispersion from different traffic peak times, Negative Binomial distribution can be used to model this behavior. For an observed traffic count y between an origin and a destination node, the NB-NLL is computed as

$$\mathcal{L}_{NB} = -\log \Gamma(y + r) + \log \Gamma(r) + \log \Gamma(y + 1) - r \log\left(\frac{r}{r + \mu}\right) - y \log\left(\frac{\mu}{r + \mu}\right)$$

for a traffic prediction $\mu > 0$ and a fixed dispersion parameter $r > 0$. Gamma (Γ) terms originate from the combinatorial normalization of the distribution, while the terms $r \log(\cdot)$

and $y \log(\cdot)$ directly involve the model’s prediction and determine how well the predicted mean explains the observed count.

For backpropagation, the gradient is calculated via automatic differentiation and flows backwards through the model to refine learned weights and biases, enforcing statistical consistency and improving traffic predictions. The loss function implementation is provided in Appendix E.

3 Experiments

To evaluate the proposed model architecture, experiments were run on real-world traffic data. The model was assessed on its ability to forecast future OD traffic flows and compared to established baseline approaches. New York City taxi trip data was used as experimental datasets, providing a realistic setting and possible practical use case. The model was realized using PyTorch (Paszke et al., 2019), trained and later tested on unseen data. Post-hoc corrections were done on the resulting predictions to minimize remaining bias and enhance results without losing the ability to predict on new data or into the future.

3.1 Setting

Real-world mobility data was used to test the models future traffic predictions in a realistic setting. Short-term and long-term traffic were predicted as total numbers of trips between taxi zones within New York City. The data was collected, aggregated, and split into training and testing periods to simulate how the model would perform in a realistic deployment setting. Unseen testing data was used to evaluate how well the model learns from data and generalizes beyond training information. The setup ensured that experiments reflect realistic fluctuations in traffic, including daily and seasonal effects, as well as irregular patterns. The model was trained on the data to learn spatio-temporal patterns on a dynamic graph.

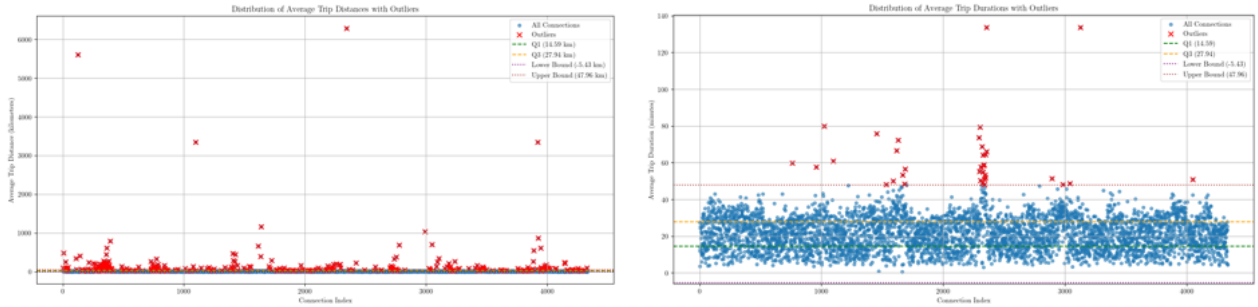
3.2 Data

The experimental dataset was derived from the official Yellow Taxi trip records published by the NYC Taxi and Limousine Commission (TLC) for the year 2024 (TLC, 2026). These records capture every licensed yellow taxi trip taken within New York City, providing detailed information such as pickup and drop-off timestamps, origin and destination locations, trip distances, and various passenger- and trip-level attributes. All locations are encoded using the TLC’s predefined taxi zones, which partition the entire city into a consistent set of geographic units. This zoning system greatly simplifies the construction of a graph by providing a fixed and interpretable spatial structure. The data was distributed as Apache Parquet files. Using a full year of data ensured full exposure to the natural variability of traffic, including weekday–weekend differences, seasonal trends, holiday effects, and irregular events. As a result, the model could learn traffic dynamics across multiple temporal scales, from short-term patterns to long-term trends. Training on such a comprehensive dataset also created

more realistic conditions for forecasting future traffic behavior, improving the robustness and generalizability of the predictive framework. Spatial adjacency was derived from the TLC taxi zones in shapefile format. The data contained 41,169,685 recorded trips across 260 taxi zones. On average, a trip lasted 17.47 minutes and covered a distance of 8.01 kilometers.

3.3 Data Pre-Processing

In a first step, the monthly files were loaded and filtered to contain trips that only began and ended in the borough of Manhattan, reducing the amount of different zones to 67 and number of trips to 34,165,990 with average trip distances of 5.24 kilometers and durations of 13.82 minutes. Using the Inter-quartile Range (IQR) method, outliers were identified if they fell below $Q1 - 1.5 \times (Q3 - Q1)$ or above $Q3 + 1.5 \times (Q3 - Q1)$ for the first ($Q1$) and third ($Q3$) quartile. The analysis found 324 trips with notably large distances (Fig. 6a). Removing outliers in relation to distance was especially important to ensure the generation of a sound and correct distance matrix. Temporal outliers were extremely rare and were identified 34 times. Due to their distribution centered mostly on specific connection as shown in Fig. 6b, they were generally not treated as faulty entries but as traffic related. More extreme outliers were correlated with the trips identified in the distance IQR analysis and removed.



(a) Distribution of Trip Distances with Outliers (b) Distribution of Trip Durations with Outliers

Figure 6: Distribution and Outlier Detection of Average Trip Distance and Duration

The data was then reduced to include only essential columns on starting and end taxi zones, times, and trip distances. To construct a graph representation, taxi zones were treated as nodes and the number of trips in a given time interval as their weighted, bidirectional edges between them. In ten-minute frames, all trips between two nodes were captured and saved as matrices. After processing of the entire year, the matrices were then stacked into a PyTorch tensor. The resulting graph shows dynamic behavior over time with changing structure.

Distances were derived directly from the "trip distance" field and calculated as the mean of

all trips between each zone pair. As urban street networks show strong spatial constraints, for example, directed roads or turning requirements, the use of real-world data allowed for more realistic geographical modeling. Additionally, traffic flow could be trained more realistically in combination with temporal information and routing decisions gained a level of influence. Distances thus did not only reflect physical properties of the city and between zones, but also observed dynamic driving behaviors.

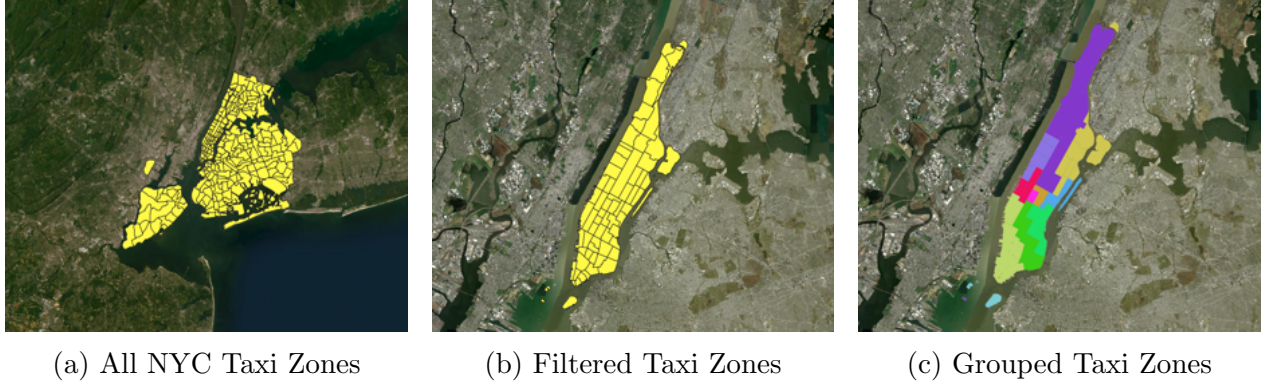


Figure 7: Geographic Distribution of Taxi Zones in Different Preprocessing Steps

The resulting graph contained 67 taxi zones within Manhattan (Fig. 7b). To simplify the graph and reduce training time, it was necessary to group nodes into larger, aggregated areas. Traffic was also highly varied, introducing noise and connections that would otherwise be disproportionally over-represented. A large number of edges did not show traffic, which would have led the model to learn unnecessary information or focus on zero-traffic connections. A denser graph allowed to focus on learning relevant traffic patterns and dominant flows.

The zones were grouped by choosing the ten nodes with highest traffic volumes over the year as initial cluster centers. Using the adjacency matrix, neighboring zones were iteratively joined into spatially contiguous units that reflect physical connectivity. If a zone was adjacent to two clusters, it was assigned to its closest neighbor based on the distance matrix, ensuring that distance was considered in clustering. Zones not adjacent to other nodes were assigned to the cluster containing the zone with the smallest mean distance to avoid creating isolated islands of taxi zones. The tensor was then aggregated by calculating the sum of all traffic of all zones within a cluster in a given time step.

The resulting graph consisted of 10 nodes connected by 29 edges with a density of 0.644. Ten additional edges were added as self-connections to symbolize traffic without each individual zone. The diameter of the graph was 3, with an average shortest path length of 1.556. The clustering process resulted in larger, aggregated zones that reflect spatial connectivity and traffic patterns while reducing noise and irrelevant connections. This allowed the model to focus on learning dominant traffic flows and patterns in a more manageable graph structure.

The average clustering coefficient of the graph was 0.6933, indicating a high level of local clustering among the nodes. The average degree of the graph was 5.8, showing that on average, each node was connected to nearly six other nodes, which is relatively high for a graph of this size and indicates a well-connected structure as it was expected for urban traffic systems. Figure 8 shows the overall structure of the graph and the corresponding centroid locations within Manhattan.

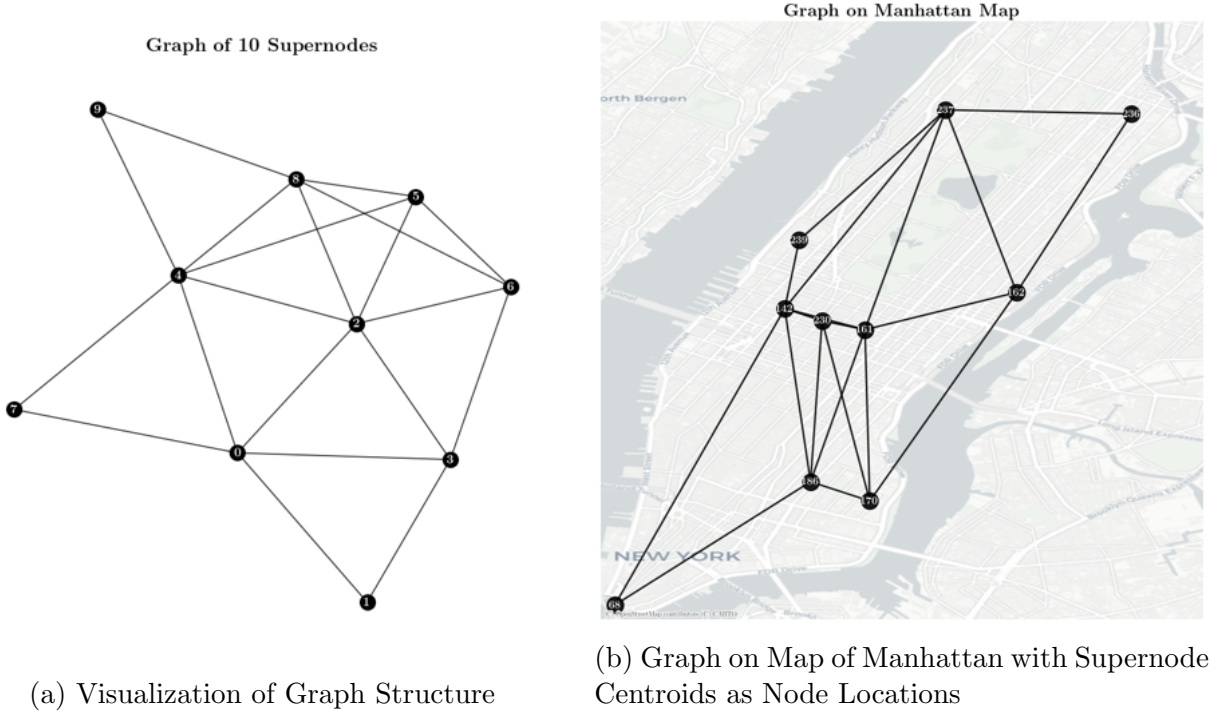


Figure 8: Abstract and Geographic Structure of the Created Graph

Node	Betweenness Centrality	Closeness Centrality	Eigenvector Centrality
0	0.1986	0.6923	0.3360
1	0.0000	0.4737	0.1417
2	0.1454	0.7500	0.4630
3	0.0889	0.6000	0.2875
4	0.2528	0.7500	0.4149
5	0.0148	0.6000	0.2932
6	0.0486	0.6000	0.3239
7	0.0000	0.5294	0.1707
8	0.0843	0.6429	0.3810
9	0.0000	0.5000	0.1809

Table 1: Node Centrality Measures

Centrality measures highlight distinct structural roles within the network. Nodes 4 and 2

(zones 230 and 237) function as the system’s dominant hubs. These achieve the highest closeness centrality scores, reflecting their strong accessibility to all other zones. Node 4 additionally records the highest betweenness centrality (0.2528), identifying it as the main intermediary through which many shortest paths pass. Node 2, in contrast, stands out through its eigenvector centrality (0.4630), indicating strong influence via connections to other well-connected zones. At the opposite end of the spectrum, nodes 1, 7, and 9 (zones 68, 236, 239) exhibit zero betweenness and low eigenvector centrality, marking them as peripheral areas with limited structural importance. Taken together, the network forms a dense structure characterized by a handful of central hubs embedded within a broader set of less influential secondary zones. Node centrality metrics can be found in Table 1.

The adjacency and distance matrices were reconstructed based on the new graph structure. Adjacency was considered if any zone within a cluster was adjacent to any node in another cluster (Fig. 9). As the resulting shapes of the new zones (Fig. 7c) are of large, irregular shapes, the shortest distance between the zones in two clusters was taken as their distance to focus on learning on the shortest possible path as the easiest connection (Fig. 10).

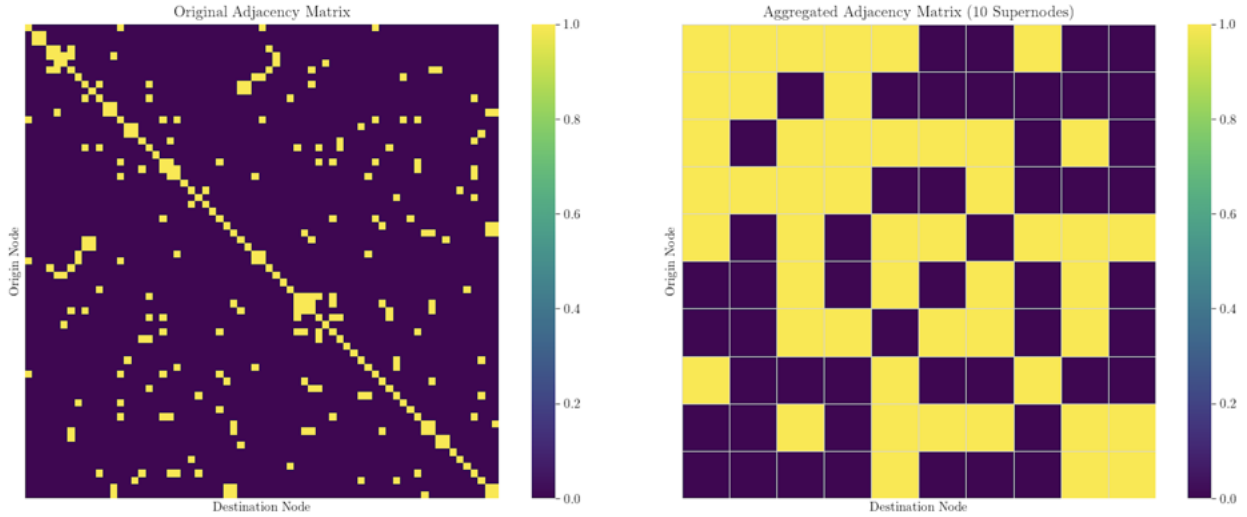


Figure 9: Original and Aggregated Adjacency Matrices

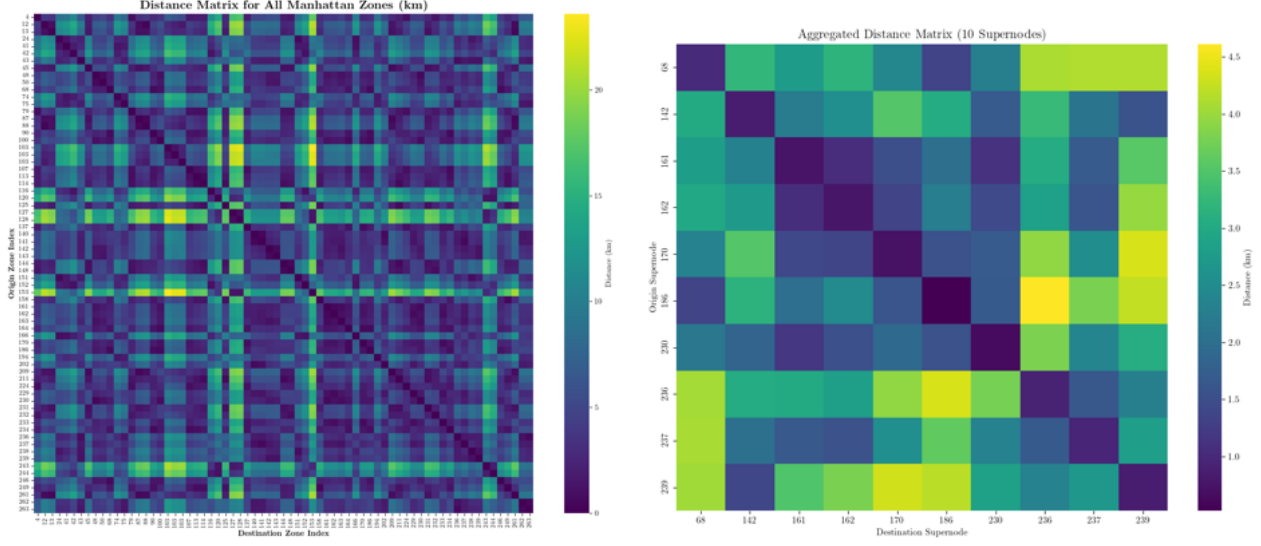


Figure 10: Original and Aggregated Distance Matrices (in kilometers)

Figure 11 shows the original sums of trips for Manhattan and for the aggregated zones. Zero-traffic connections have been eliminated from the graph and noise reduced, zones show distinguishable amounts of traffic and visual differences.

The tensor was split into two parts. For training, the first 80% of days in the data were used. Tests were conducted on the learned weights and the remaining 20%.

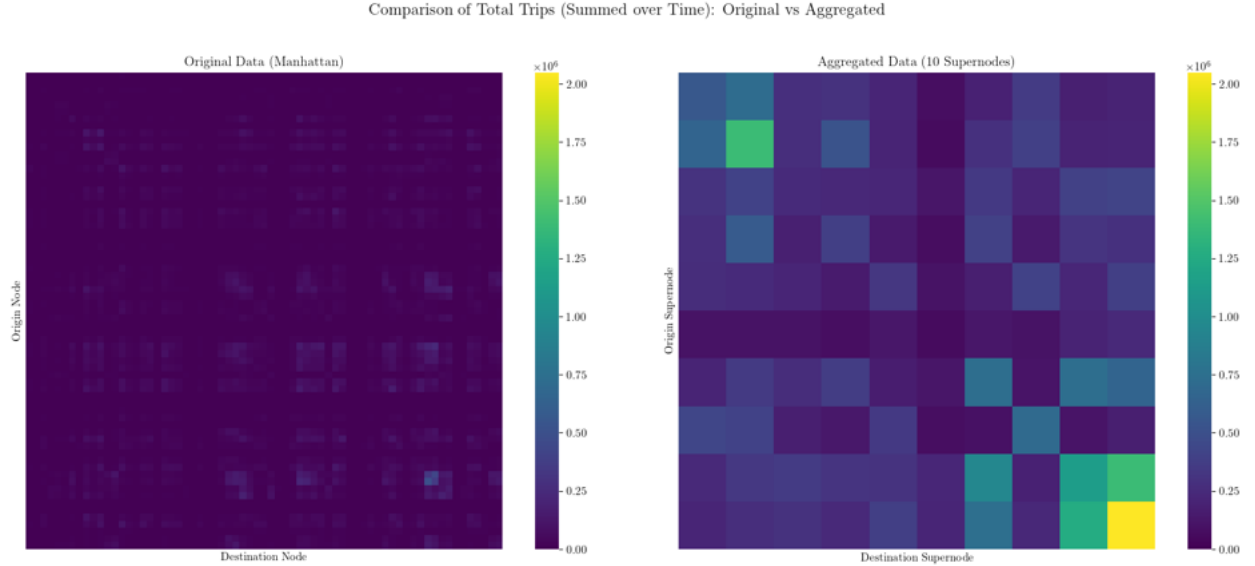


Figure 11: Original and Aggregated Trip Sums per OD Connection

3.4 Metrics

Mean Square Error

To measure the distance between true and predicted values while penalizing large deviations, mean square error (MSE) was used. By emphasizing large error more strongly, MSE helps to identify significant outliers and influence on prediction mistakes. For n observations, the MSE between a true value y_i and a predicted value $\hat{y}_i$ is calculated as:

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Root Mean Square Error

For easier interpretation and assessment of prediction metrics, root mean square error (RMSE) was analyzed. Keeping the benefits of MSE, it restores the original unit of measurements. RMSE is calculated as:

$$RMSE = \sqrt{MSE}$$

Mean Absolute Error

The mean absolute error (MAE) was used to further evaluate the accuracy of model predictions. It is a measurement mirroring the average absolute difference between true and predicted data and gives results in the unit of data measurements, in this case the number of OD trips. Additionally, it does not weight errors based on their magnitude, thus giving a generally good overview of where and in which way the model predicted traffic on the graph. MAE gives a good overview of mis-predictions in the model. It is calculated as:

$$MAE = \frac{1}{N} \sum_{i=1}^n |y_i - \hat{y}_i|,$$

where $\hat{y}_i$ is the predicted value to a true value y_i and n the number of observations in data.

Paired t-Test

To test the model for systemic over- and under-predictions, true and predicted data were compared using a paired t-test. Testing each pair of samples in the dataset, the mean difference between samples is checked for its significance and possible consistent mis-predictions identified. It tests for possible systemic bias in the model, ignoring the error size. The mean difference between the datasets gives an indication of such errors with positive values indicating consistent over-predictions, while negative results mean consistent under-predictions. The t statistic calculates the standardized mean difference in relation to variability and sample size. A large value would indicate a large underlying bias in predictions. Additionally, the p-value is calculated. It gives the probability of observing the mean difference in data under the assumption of identical datasets. A small p (< 0.05) would mean measured differences are likely due to model bias. As the p-value shrinks with the sample size, additional metrics are necessary to interpret it. For this, Cohen's d is observed to identify the practical impact of p. It compares the means of two datasets in relationship to their standard derivations. A negative value indicates a larger mean in the second group, whereas a positive result shows the opposite. The value of Cohen's d describes the difference in datasets as multiples of means. In the context of traffic prediction, this can help putting the p-value into context. Due to naturally high amounts of samples in the data, a very small mean is often the result. While this can be a sign of severe bias in the model, measuring Cohen's d can put it into context of practical consideration. While the observed difference between the datasets might not be reducible to standard deviation, a very small bias can be ignored in practice.

Cosine Similarity

Cosine similarity is used to compare two vectors' similarity by calculating the cosine of the angle between them. It is calculated as:

$$CosineSimilarity = \frac{Y \cdot \hat{Y}}{\|Y + \epsilon\| \|\hat{Y} + \epsilon\|},$$

where Y and $\hat{Y}$ are vectors representing true values and predictions, respectively. $Y \cdot \hat{Y}$ is the dot product, and $\|Y\|$ and $\|\hat{Y}\|$ are the vectors' magnitudes. As zero-values are inherent to the data, $\epsilon = 1e-8$ is added to avoid division by zero.

The result ranges from -1 , representing opposite direction of vectors (180° angle), to 1 , showing same direction (0° angle). A value of 0 describes an orthogonal relationship between

vectors, meaning no directional similarity is given. Cosine similarity ignores the vectors’ magnitudes and does not give an indication on flow volumes. Instead, it shows the accuracy at which patterns could be learned and reproduced. As OD traffic flow can be represented as vectors between two nodes, it is used as a measurement for assessing the model’s performance in predicting the spatial distribution of traffic over time. For deeper insights, cosine similarity is calculated row and column wise. Information on destination distribution for origin nodes can be gathered from rows, showing the accuracy at which traffic flow from one node to its destination was learned. Columns show the in-flow into a node from different origins. For analysis of OD predictions, rows and columns from $N \times N$ matrices are considered as N -dimensional vectors, representing a node on the graph.

3.5 Training

Table 2: Model hyperparameters and training configuration

Parameter	Value
Number of vertices	10 (inferred from data)
Chunk size	36
Number of chunks (GRUs)	4
Short Window length (GRUs)	36
Long Window length (LSTM)	144
Hidden size	64
Input features per node	1
Learning rate	1e-3
Training epochs	50
Batch size	64
Optimizer	Adam
Loss function	Negative Binomial NLL Loss
Weight Initialization	Xavier Uniform
Scaler	GradScaler

The model was training on the first 80% of data, representing approximately the first 292.2 days of the year 2024. Training data consisted of 42,170 ten-minute OD matrices, containing a total of 26,810,210 trips. The proposed architecture was initialized with one long-term branch receiving 144 consecutive matrices, representing a full, single day’s traffic. In parallel, four short-term branches were trained on 36 matrices of the same long-term window each, allowing the model to capture short changes in traffic and daily trends simultaneously. The GRUs received the matrices for the same six-hour spans of each day, reducing the complexity of patterns while also allowing to learn rapid changes more effectively. Training the LSTM

to learn daily patterns enhanced forecasting capabilities on a day-by-day basis and weekday-weekend differences in traffic patterns. Both the next step outside the window, as well as a series of temporal horizons at longer temporal distances were predicted simultaneously.

Training was performed for 50 epochs with a batch size of 64, using the Negative Binomial NLL loss and Adam optimizer (Kingma and Ba, 2014). All units used a hidden size of 64 with a single input feature per node. A learning rate of $1e-3$ was chosen for stable convergence. To ensure efficient optimizations, GradScaler was used for mixed-precision training. Weights were initialized using the Xavier uniform distribution (Glorot and Bengio, 2010). Table 2 provides the full list of hyperparameters.

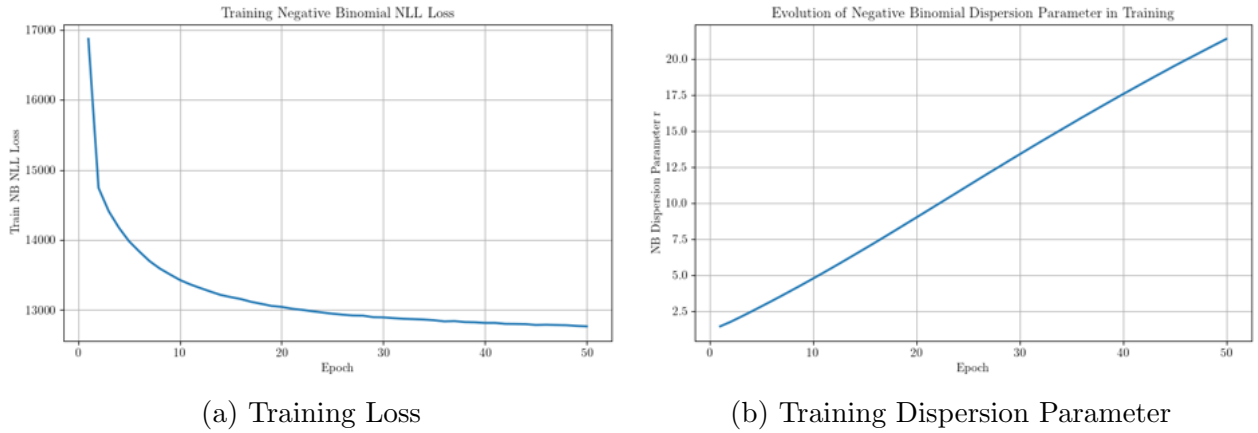


Figure 12: Evolution of Loss and Dispersion Parameter r during Training

In training, the model showed a sharp initial decrease in loss and smooth convergence in later epochs (Fig. 12a). The model rapidly learned predicting traffic volumes and refinement continued gradually. The dispersion parameter was continuously adjusted to the over-dispersed data to reduce variance and allow the model to capture variable amounts of traffic over time (Fig. 12b).

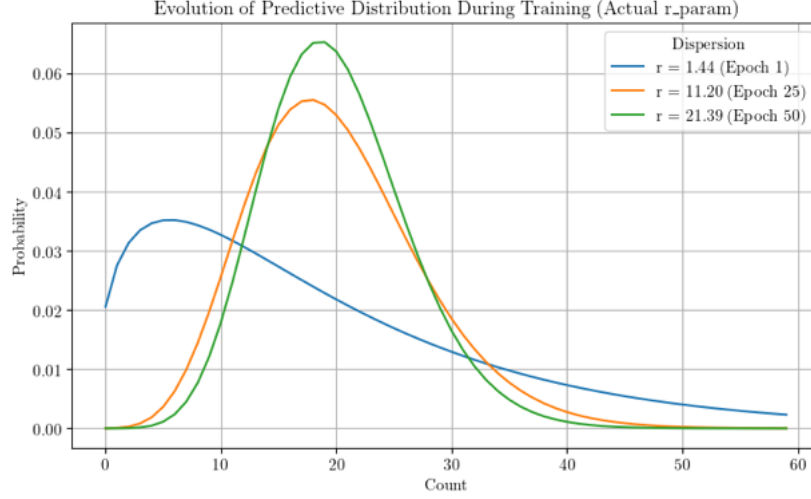


Figure 13: Change of Distribution in Training

The predicted distribution changed from a broad to a more concentrated and calibrated form during training (Fig. 13). Through refinement of the dispersion parameter, variance was progressively reduced and unnecessary noise removed from the predictions. Early epochs displayed a skewed, heavy-tailed distribution, indicating strong issues in prediction certainty. As training continued, the distribution became more concentrated and even. The model learned a stable probabilistic representation of the data, as shown by the sharply peaked distribution in epoch 50.

4 Results

After training, the model was tested on the remaining 20% of data, totaling 7,355,884 trips across 10,543 matrices. The testing tensor was loaded and the model applied using weights learned in training. Initial predictions were analyzed for volume and flow errors and comparisons made to baseline models. To address systemic bias, post-hoc corrections were made to the predicted matrices.

Overall performance and next-step predictions were analyzed by comparing results produced by the model and corresponding ground-truth data. To assess the model’s ability to predict traffic over extended horizons, forecasts were generated at different distances. Predictions were made on matrices at a distance of one matrix (10 minutes), 36 (6 hours), 144 (1 day), 432 (3 days) and 1008 (7 days) matrix horizons. Long-term predictions were again checked for their accuracy in volume and flow estimation and compared to baselines.

4.1 Single-Step Performance

Table 3: Forecasting Errors

Mode	MAE	MSE	RMSE
Single-Step	2.6880	24.0513	4.9040

The model showed high accuracy in predicting immediate future traffic. Errors remained low with an MAE of 2.688, MSE of 24.051, and RMSE of 4.904. These results indicate that the model produced accurate short-term traffic estimates. Low differences between predictions and true data exist and are maintained throughout predicting a series of OD matrices. The errors reflect stable performance for next-step predictions.

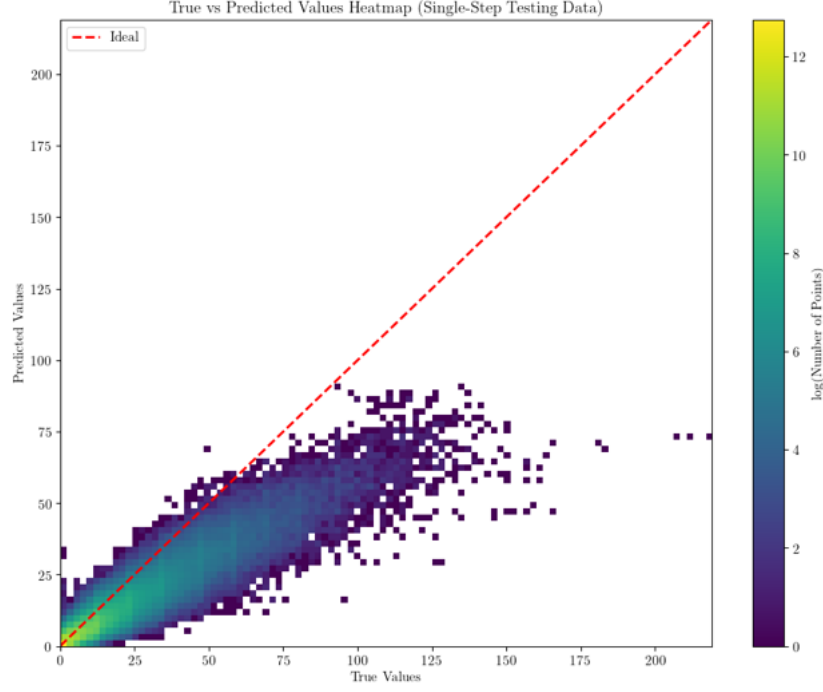


Figure 14: Scatterplot heatmap of true and predicted values

Figure 14 shows the matching of true values in predictions. The model learned the overall trend but shows significant underprediction. Especially larger values fall below the ideal diagonal, indicating calibration issues for higher traffic volumes. Error increases with larger magnitudes and the spread widens accordingly, suggesting issues with maintaining accuracy with growing traffic. The model learned the general trend but failed to preserve the correct scale. The pattern points to structural bias in the model. The consistent underestimation indicates the model favoring very low volume predictions. As a result, high-traffic peaks and node are underestimated more strongly. The plot indicates problems in the model's calibration to observed traffic numbers.

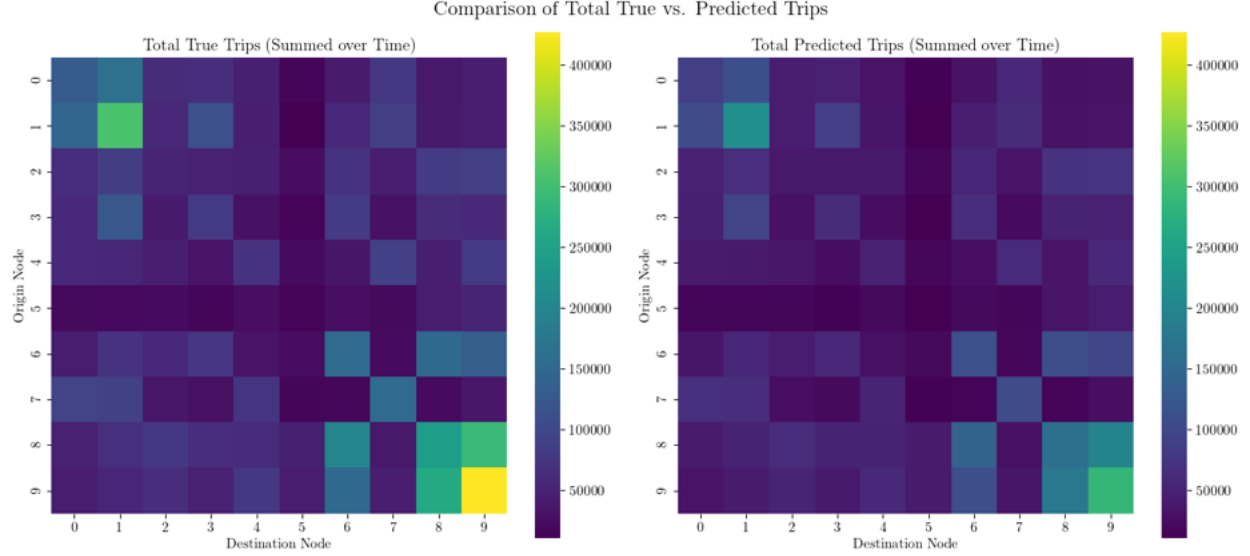


Figure 15: Heatmaps of true and predicted traffic sums

This bias can be seen further in Figure 15. The model matched the overall structure of the OD matrix, but strongly underestimated. While all traffic is underestimated, especially high-traffic connections show visually less traffic. Although spatial patterns are preserved, the predicted sums appear uniformly shifted, indicating system down-scaling across all OD pairs. The comparison suggests that the model learned spatial relationships between nodes but struggled to transform these into the correct magnitudes. The underestimation follows relative traffic sums but loses scale.

Table 4: Paired t-test Results and Effect Sizes

Analysis Level	t-statistic	p-value	Cohen's d	Mean Difference
Overall	416.1663	0.0	0.4081	1.8531
Row-wise (Average)	139.8588	0.0	0.4337	1.8531
Column-wise (Average)	138.5727	0.0	0.4297	1.8531
Cell-wise (Average)	45.8195	0.0	0.4493	1.8531

Table 4 confirms systemic underprediction tendencies in the model with respect to traffic volumes. The extremely high t-statistics show relevant differences between the true and predicted trip numbers and p-values of 0.00 this being due to model bias. For all levels, there are strong indications of forecasting issues originating from the model. Consistent mean differences and values for Cohen's d further show global and meaningful bias. Structural issues persist in the model and errors are not subject to isolated anomalies in data.

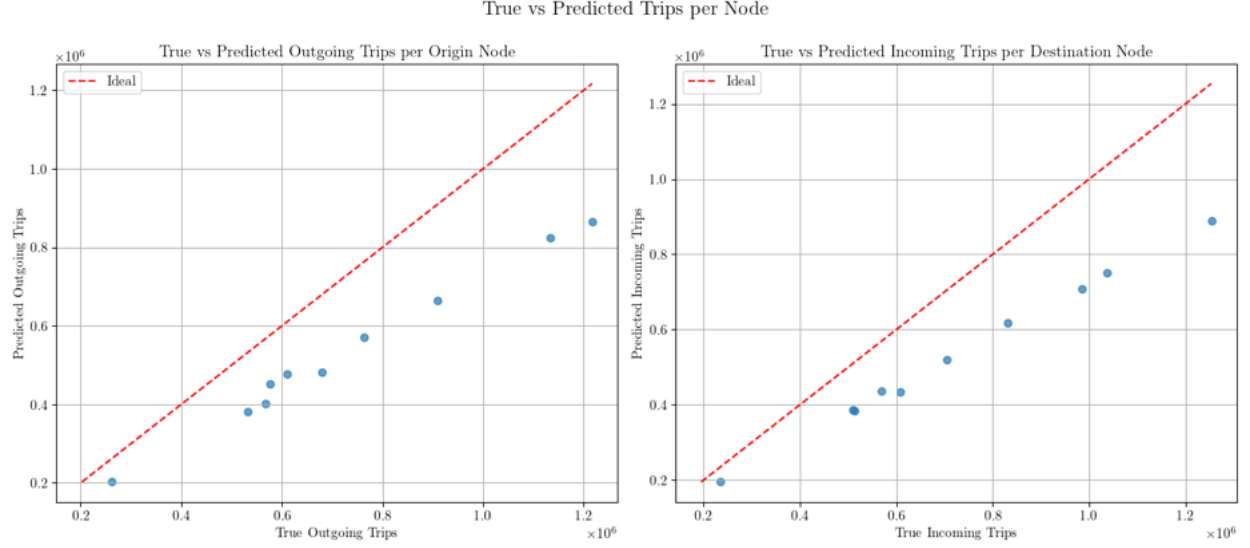


Figure 16: True vs predicted values per node

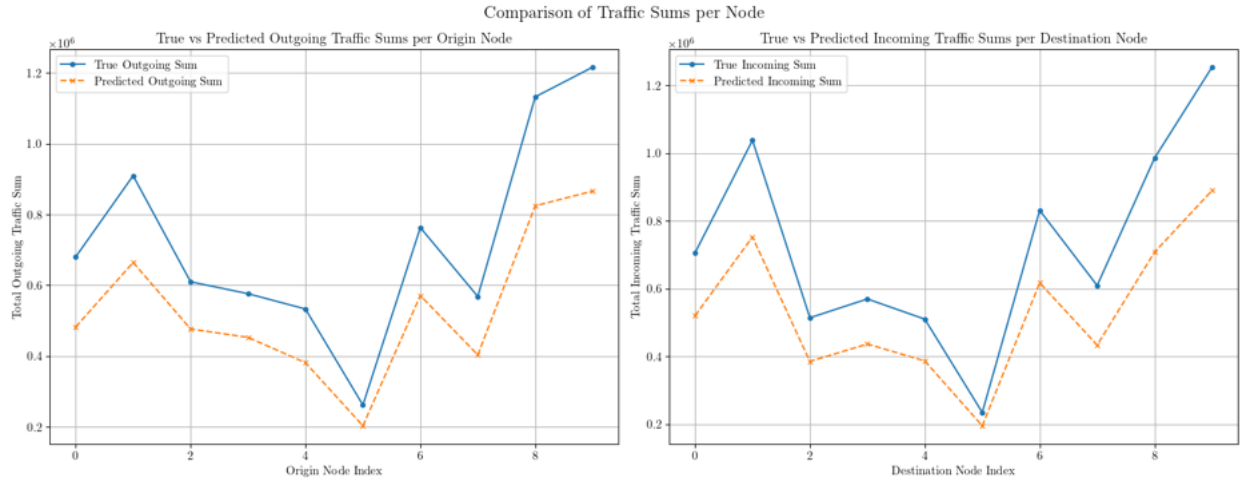


Figure 17: True vs predicted traffic sums per node

Figures 16 and 17 visualize the problem of underprediction at the node level and make the model's systemic bias visible. In Figure 17, the scatter plot shows all predicted traffic volume fall below the ideal diagonal. Model outputs are consistently lower than true observations across all nodes. They form a straight line, indicating global and systemic bias to underestimations rather than singular errors. Figure 17 puts emphasis on this observation. While relative traffic was matched for most nodes, the graph showing absolute estimates is shifted to lower values. The magnitude of this downward shift is not fully uniform, but correlates with the amount of traffic, as shown by the growing distance between true and predicted volumes with increasing traffic sums. The model captures spatial importance of nodes, but systemically loses scale. The model seemingly learned to represent spatio-temporal patterns

but introduced bias while mapping learned features to traffic counts. Further, it shows that it learned general patterns but failed to calibrate in regression layers or loss weighting.

Table 5: Per-Matrix Cosine Similarity Statistics

Rows Mean	Columns Mean	Overall Mean	P50	P75	P90	P95
0.8802	0.8869	0.9052	0.9559	0.9686	0.9753	0.9787

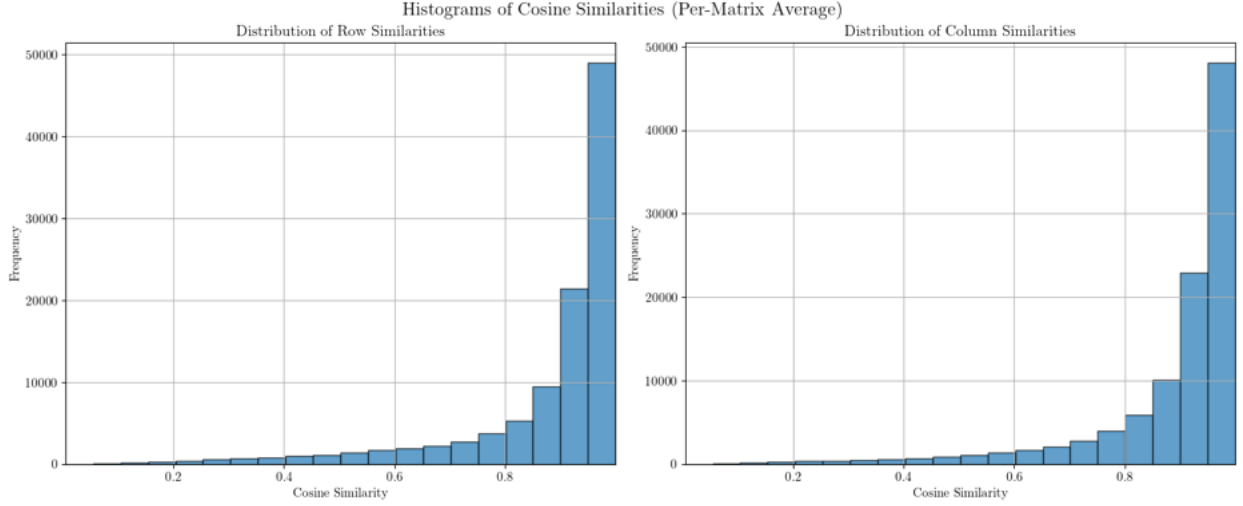


Figure 18: Cosine similarities for rows and columns

Figure 18 shows the distribution of cosine similarities for outgoing traffic in rows and incoming traffic in columns for the nodes. The overall very high values show the model’s ability to accurately capture traffic flows, as the vast majority rows and columns show very strong similarities. The high frequency of values close to 1 indicates that the model consistently learned directional structures in the graph. Predicted traffic flows match the same temporal patterns as present in the recorded data. The model not only learned traffic volumes at certain times but reproduced the underlying spatio-temporal relationships.

This is further emphasized by Table 5. Row-wise and column-wise means of 0.8802 and 0.8869 show high accuracy in preserving directional flows over time, while the overall mean of 0.9052 indicates learning of global patterns. The very high median value (P50) of 0.9559 and upper percentiles of 0.9753 (P90) and 0.9787 (P95) further show near-perfect cosine similarities in the majority of matrices. The model maintained temporal patterns and generalized across nodes and time periods at great detail.

Although consistent tendencies to predict overall lower traffic volumes, the cosine similarities confirm the model’s ability to learn fine-grained temporal structures. The shape and structure of traffic flows was captured, showing that the pattern-learning mechanism is effective, while volume predictions require further refinement. As underpredictions appear throughout all

nodes and times at almost equal levels, post-hoc corrections can help combating these problems without restructuring the model or additional training. A possible mitigation strategy to overcome this scaling issue is shown in section 4.3.

4.2 Baseline Comparison

In order to put the model’s performance into perspective, it was tested against a series of established baseline models used for traffic prediction. Predictions on the same true data were calculated for persistence, a historical-average, and for a seasonal naive baseline model and the results compared.

Persistence Baseline

The persistence model assumes that the next value in a timeseries is identical to the most recent observation in true data as

$$\hat{y}_{t+1} = y_t$$

It captures short-term continuities, providing strong results if traffic evolves gradually and slowly over time.

Historical Averages Baseline

For historical-averages, next value is predicted as the mean of all previously observed true values. Predictions are calculated as

$$\hat{y}_{t+1} = \frac{1}{t} \sum_{i=1}^t y_i$$

Long-term trends are modeled at the loss of detail and short-term fluctuations.

Naive Seasonal Baseline

The naive seasonal model processes data in a defined window length w . A value is predicted by repeating the value at the same position as observed in the window before:

$$\hat{y}_{t+1} = y_{t+1-w}$$

It captures reoccurring patterns in the data, modeling daily traffic patterns in periodic structure.

Short-Term Results

Table 6: Next-Step Forecasting Error Comparison per Timestep

Method	MAE	MSE	RMSE
Model	2.6880	24.0513	4.904
Persistence	5.3508	70.0514	8.3697
Historical Average	5.0637	73.4841	8.5723
Seasonal Naive	3.2533	33.1084	5.7540

Table 6 shows the errors for the baselines in comparison. The model showed over all lower errors than all of the baselines, with strong improvements over the persistence model and historical-averages. The seasonal naive baseline shows the best results of all baselines, with only slightly worse results than the model. The plots in Figure 19 give an indication of the applicability on traffic prediction. While the model shows systemic bias, the results are more reliable. Both the persistence and historical-average baselines lack the capability to correctly predict traffic. The seasonal naive appears more centered along the ideal diagonal, the variance and wide spread seen in data points, however, make it unsuitable for traffic predictions. Accuracy varies widely due to repetition of patterns. If traffic deviates from earlier observation, the seasonal naive introduces large uncertainty and error.

While model predictions show systemic bias, points appear as more packed in the scatterplot. Errors and variance exist, but are much less drastic than they are in the baselines, showing more reliable predictions.

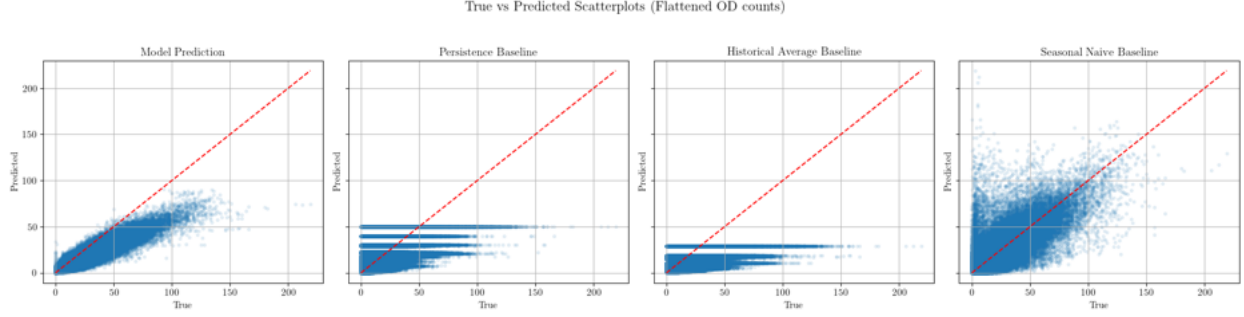


Figure 19: Scatterplots for Model Predictions and Baselines

Further problems with prediction reliability can be seen in Figure 20. The model shows the lowest MAE over all time steps, followed by the seasonal naive. For the proposed model, MAE slowly increases until it rapidly drops to a lower value again, indicating slow loss of accuracy before stabilization. Stronger inconsistencies can be found for the seasonal naive model. The rapid changes in error show the dependence on seasonal patterns, which lead to mispredictions if not met.

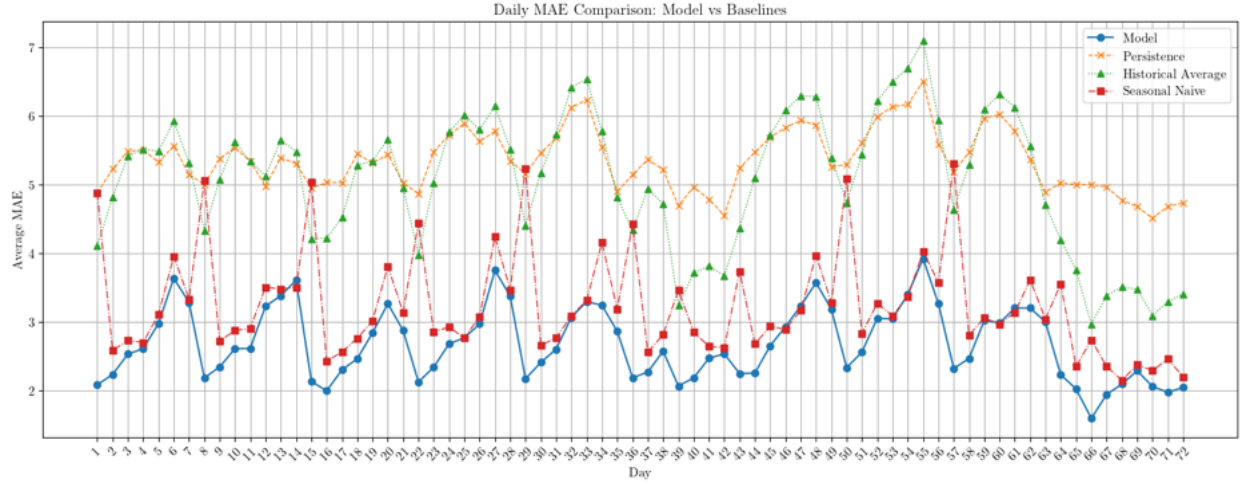


Figure 20: MAE Over Time for Model and Baselines

Figure 21 shows the traffic sums per day. The model shows consistent underpredictions but strong matching of patterns in traffic per timestep, showing its capability to effectively learn traffic patterns even if bias in scale exists. Overall, fluctuations are met at great detail and peaks and troughs are aligned. The model shows its advantages in capturing short-term fluctuations and more consistent long-term trends. In contrast, the repetition of patterns from the seasonal baseline leads to issues when drastic changes occur. It fails to capture abrupt shifts or irregular events, leading to mismatches during periods of unusual traffic.

The historical-average and persistence baselines perform even less effectively. Both methods produce flat predictions that are incapable to reproduce variations in traffic.

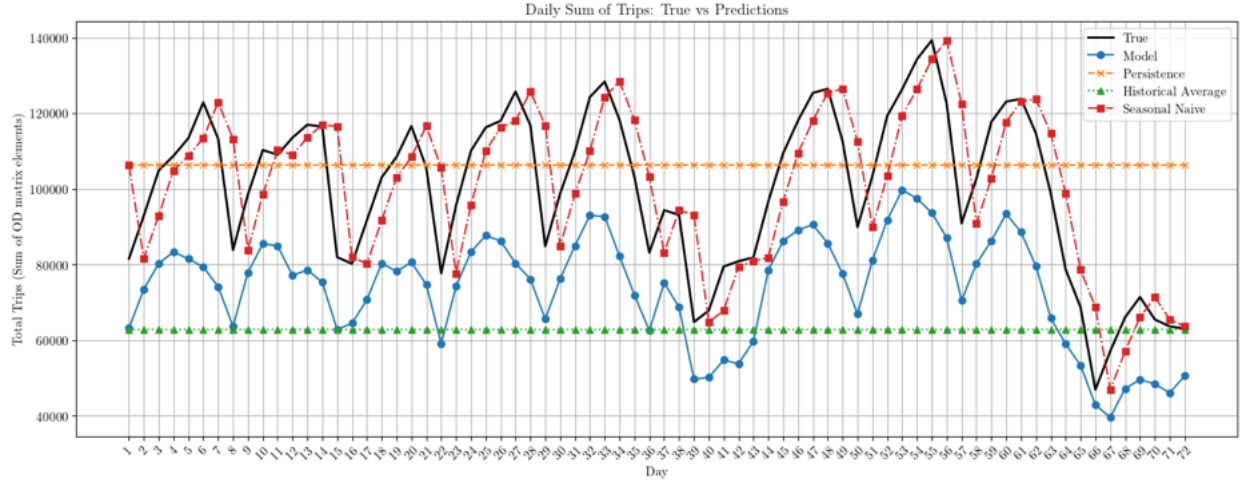


Figure 21: Total Number of Trips per Day for Model and Baselines

4.3 Post-hoc Correction

As the metrics show systemic and consistent underprediction, the use of post-hoc corrections on the predicted data could help achieving more accurate results. For a first naive correction, the global bias was calculated and all predicted values shifted by it to account for possible constant bias. Additionally, a residual correction based on linear regression was applied. By training a simple residual model on predicted data samples, systemic bias was captured.

Global Bias Correction

To calculate a possible global bias in the model, the residuals $e_t = y_t - \hat{y}_t$ were calculated for each prediction $\hat{y}_t$ of true value y_t . The bias was then calculated as

$$b_{global} = \frac{1}{N} \sum_{t=1}^N e_t$$

The result is a constant factor that represents the average distance between a true value and its prediction. All predictions are then shifted by adding it to each $\hat{y}_t$ as $\hat{y}_t^{corr} = \hat{y}_t + b_{global}$

Residual-based Linear Regression

As the assumption that bias is the same for all data points is unlikely given the distribution between true and predicted values in Figure 14, a linear regression based on residuals could help achieve more accurate bias capture. A linear model is trained to approximate e_t as

$$e_t \approx r(\hat{y}) = a\hat{y} + b$$

with the corrected prediction calculated as

$$\hat{y}_t^{corr} = \hat{y}_t + r(\hat{y}_t) = \hat{y}_t + a\hat{y}_t + b.$$

Results

Table 7: Improvement of Corrected Models over the Raw Model

Method	MAE	% Imp.	MSE	% Imp.	RMSE	% Imp.
Model (Raw)	2.6880	—	24.0513	—	4.9042	—
Model (Global Bias)	2.8603	-6.41%	20.6175	14.28%	4.5407	7.41%
Model (Residual)	2.2301	17.03%	12.7961	46.79%	3.5772	27.05%

Table 7 shows strong improvements achieved by post-hoc correction. Both the application of a global bias and residual linear regression helped reducing issues with outliers in the data. Residual-based linear regression shows strong improvements for all metrics, enhancing the model’s predictive performance drastically. MAE was lowered by 17.03% to 2.23 and strong reductions in MSE and RMSE show additional improvements over the raw model. The average MSE was nearly halved, while RMSE similarly improved by 27.05%, showing reductions in large-magnitude errors. Day-level improvements can be seen in Figure 22. Although the error pattern remains, clear reductions in MAE are visible and high-error peaks are significantly reduced.

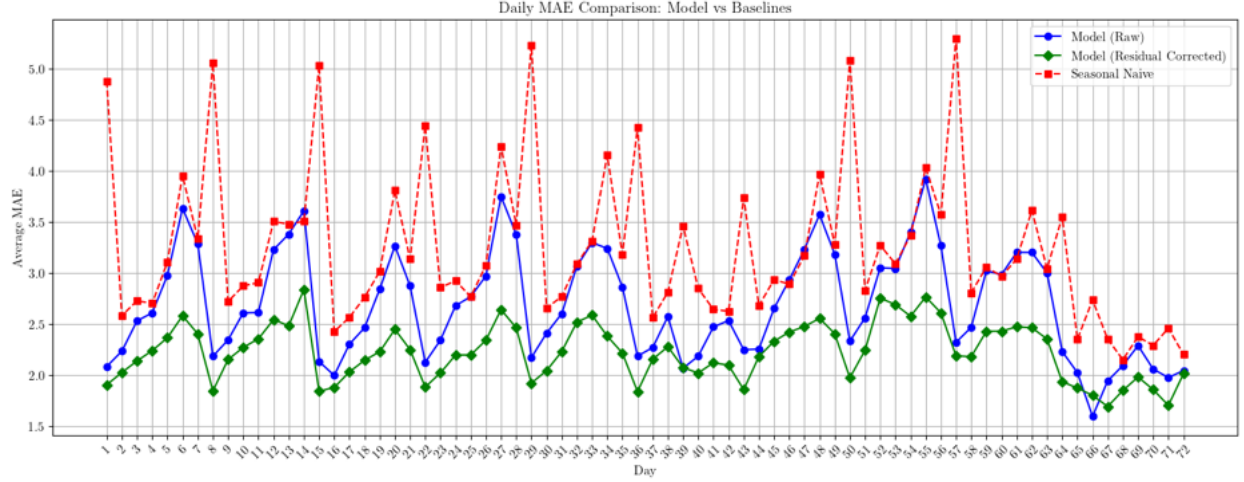


Figure 22: MAE Over Time for Corrected Model and Baselines

Figure 23 further shows how post-hoc correction improved the model's results. The residual corrected model lost most of the systemic underpredictions observed before. The scatter plot aligns more strongly with the ideal diagonal. Predictions now match true results to a larger degree and, while over- or underpredictions still occur, the results are more stable across scales.

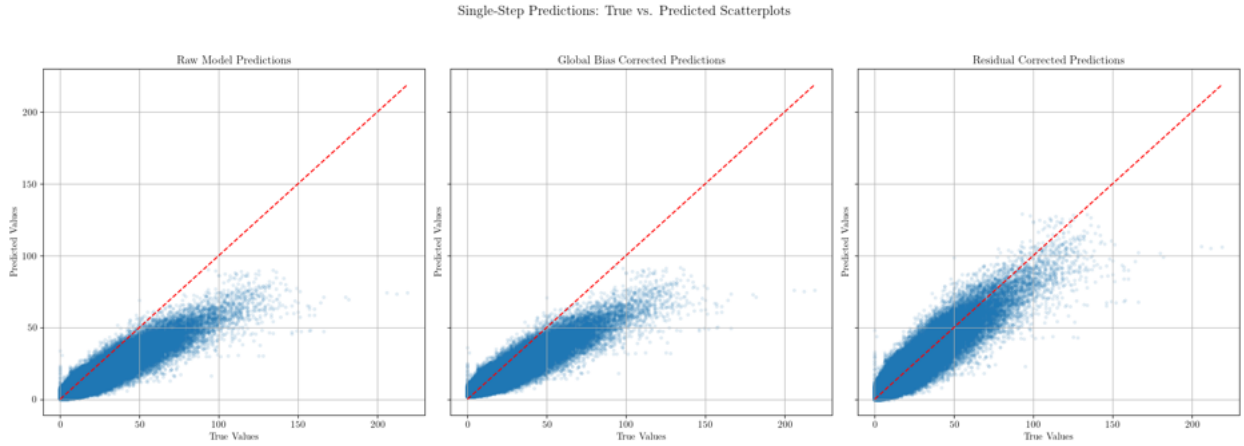


Figure 23: True vs predicted values after correction

The model now shows near-perfect predictions in traffic sums over time (Fig. 24). Earlier underpredictions disappeared and predicted traffic is almost identical with ground truth data. It further shows the model's performance in predicting not only patterns correctly, but matching traffic volumes.

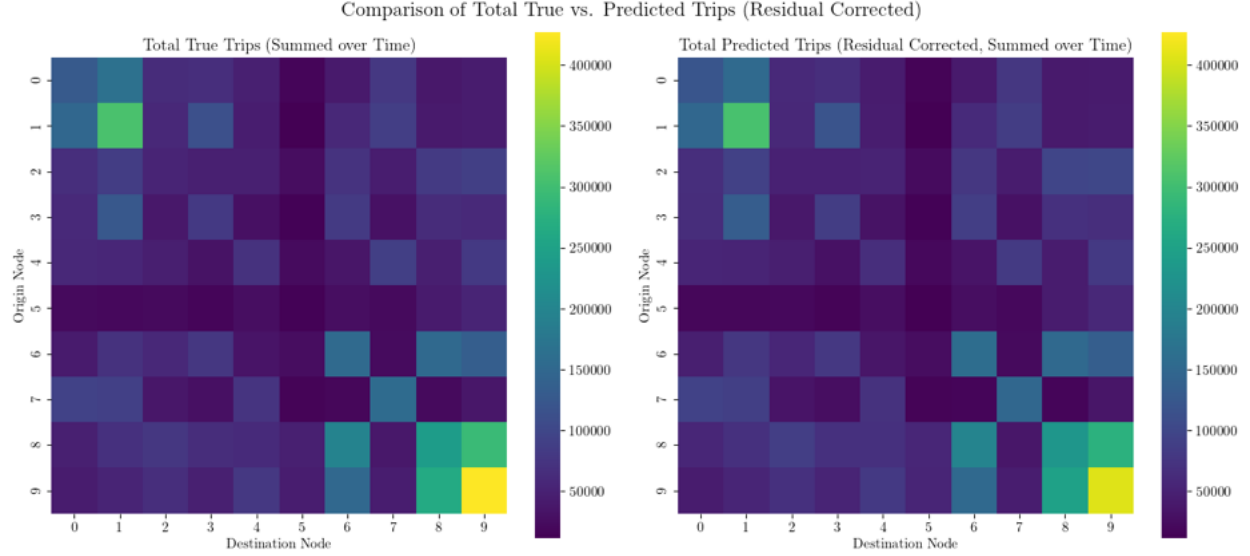


Figure 24: True and Predicted Traffic Sums

The improvement was largely achieved because the correction explicitly accounted for how prediction errors varied with the magnitude of the forecasts. The linear residual model captured systematic, scale-dependent bias patterns that the original model failed to learn. Using a simple linear regression also ensures that the correction can be applied to future, unseen predictions without requiring ground-truth values. Since the regression relies solely on the mapping between raw and corrected predictions, no additional observations are needed, making the approach practical for real-world forecasting and enhancing the model’s generalizability. Because the correction only adjusts the predicted traffic volumes, cosine similarities remain unaffected and learned patterns of traffic flow are retained. The combination of highly accurate flow predictions and improved traffic magnitude estimates demonstrates the model’s improved capability to forecast future traffic.

Correcting predictions of unseen data was simulated by calculating the regression function on limited amounts of the testing data. Remaining predictions were then corrected and compared to true matrices.

Table 8: Effect of Training Data Length on Forecasting Performance

Mapping Data Length	MAE	MSE	RMSE	Bias (%)
—	2.6880	24.0514	4.9040	−26.6
1 day	2.2511	13.7670	3.7104	−5.2
7 days	2.2381	12.8423	3.5836	+1.3
10% of data	2.2393	12.8636	3.5866	+1.2
25% of data	2.2295	12.7857	3.5757	−0.7
50% of data	2.2290	12.8230	3.5809	+0.6

Small amounts of data already helped overcoming bias in the model to a significant degree. Table 8 shows a comparison of residual-corrected predictions using different amounts of data for the linear mappings. At one day of data the results were already significantly enhanced. The MAE was reduced by 17% to 2.511 and bias was drastically reduced. Further improvements can be seen with larger amounts of regression data. The earlier observed bias is not inherent to the spatio-temporal representations learned by the model but can be mitigated by simple post-hoc calibrations. The progressive reduction in error further shows that residual patterns in predictions are highly learnable and consistent over time, allowing the linear model to compensate model bias. A small calibration dataset can help calibrating the model outputs without changing the architecture. This is especially helpful for ongoing deployment for traffic forecasting. Initial bias can be corrected from a single day of training data reserved for bias correction. As time progresses and recorded data becomes available, future results can steadily be used for improved post-hoc calibration.

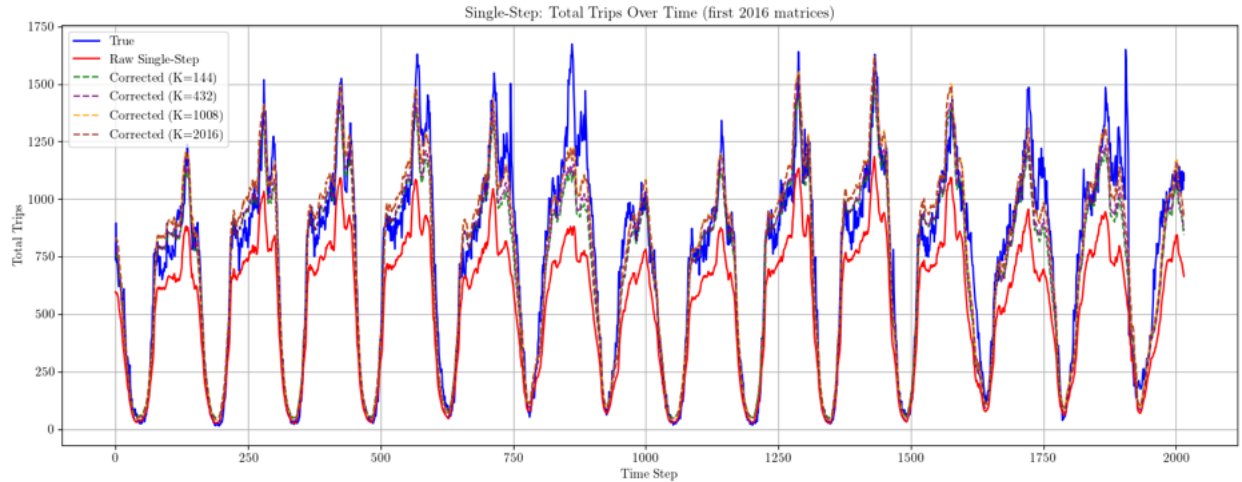


Figure 25: True and Predicted Traffic Sums per Matrix over Time

The effect of different sizes of training data for the linear regression is visualized in Figure 25. The predicted traffic volume shows significantly higher accuracy. The curves align with the

true data to a very high degree, showing that the correction helped minimize overall systemic bias.

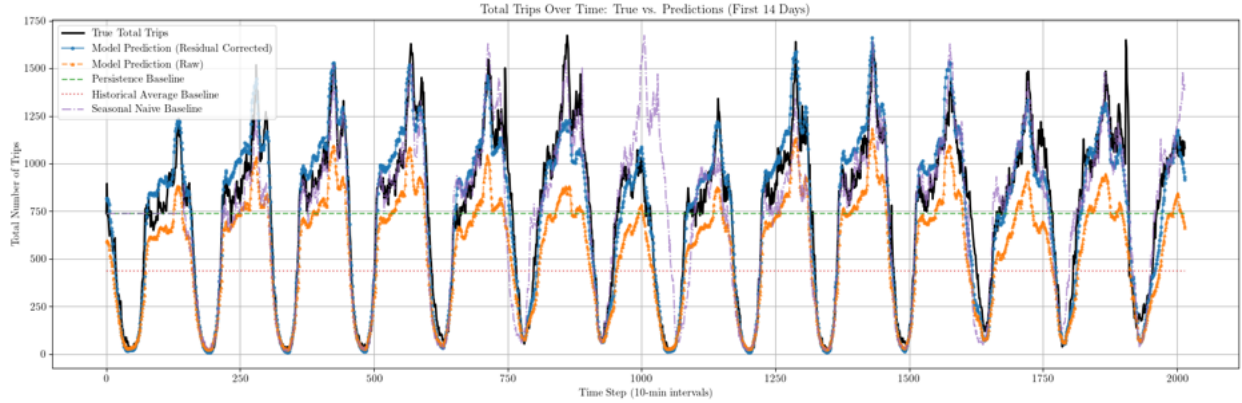


Figure 26: Comparison of True and Predicted Traffic Sums per Matrix over Time

Lastly, residual corrections further increased the model’s predictive performance over the baselines. Traffic volumes shown in Figure 26 line up more accurately with true data than baseline results. While the seasonal naive shows extreme deviations, these are not present for the model. Compared to the raw model output, the residual mapping reduces bias and restores scale without affecting the temporal structure.

4.4 Long-Term Predictions

For long-term forecasting, the model predicted the step at $t + d$, that is, the OD matrix at a fixed number of steps in the future defined by a temporal horizon d . Based on current traffic and specifically learned weights, trips at six hours, one day, three days, and one week in the future were estimated. Every further matrix of training data was read and the next step at the distance predicted. To generate these forecasts, the model processed each input window independently and calculated direct estimates of a future matrix without learning its own estimates in later steps. Rather, the model produced single-step estimation at greater distances. For each horizon, the model processed a long-term representation from recent observation at a specific point in the future. Different horizons were processed in parallel as separate learning targets, learning short-term traffic at a local, immediate level and projecting it into future trends. Results were not corrected, to simulate realistic deployment on completely unseen data before making predictions.

Table 9: Multi-Step Forecasting Performance Compared to Baseline Models

Model	Horizon (10-min)	MAE	MSE	RMSE
Single-Step Model (raw)	1	2.6880	24.0513	4.9042
	36	2.5266	20.6940	4.5491
Multi-Step Model	144	2.5499	18.5864	4.3112
	432	2.7268	22.6039	4.7544
	1008	2.7651	22.6454	4.7587

Table 9 shows the model’s performance across different longer time spans. Even at shorter horizons, long-term predictions show higher accuracies than in the raw single-step mode. Error metrics are very low across all temporal distances, with MAE ranging from 2.53 to 2.77 and RMSE values between 4.31 and 4.76. While errors grow for longer horizons, this growth is minimal. These narrow differences suggest that the model shows strong capabilities to learn broader temporal dependencies in the data and does not only rely on short-term patterns. The small worsening in predictions is especially notable given the growing distances on which predictions are made. The lower MSE at the daily level suggests positive influence of the LSTM on future forecasts as its window length might benefit predictions on the daily level, whereas horizons of under 144 rely heavier on the GRUs weights.

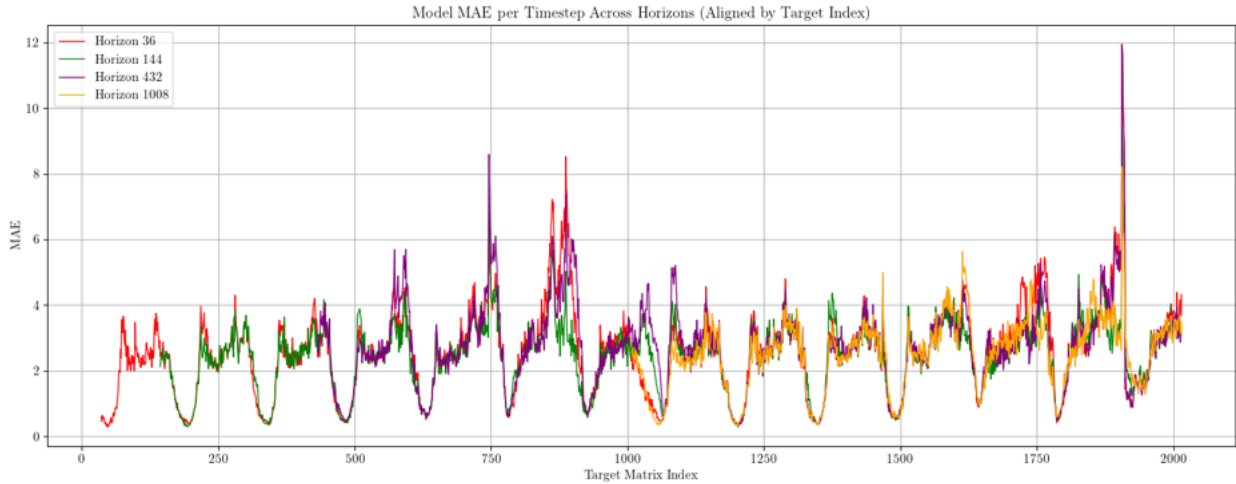


Figure 27: Model MAE per Horizon

This observation is emphasized by Figure 27. The differences between MAE over time across different horizons is minimal and differences limited to occasional outliers. The model learned to predict traffic across on different temporal levels at similar degrees. Minor improvements are visible for horizons of 144 and above. The joined temporal modeling of LSTMs and GRUs at this stage show slightly smaller errors than the 36 matrix horizon.

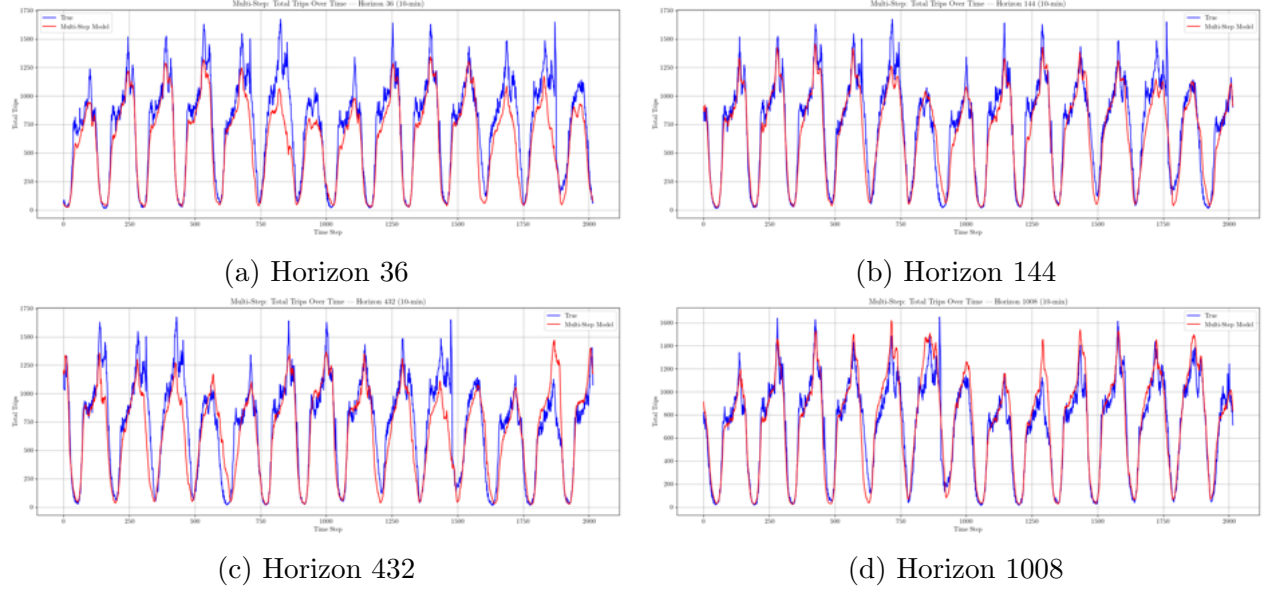


Figure 28: True and Predicted Traffic Sums per Forecasting Horizon

This improvement is further visualized by the comparison of total traffic volumes in Figure 28. Even at the shortest horizon, the model manages to predict traffic volumes with high accuracy. The curve follows true data in most cases and only introduces error in sudden peaks or drops. The scatterplot in Figure 29 confirm this observation. While the overall larger spread indicates general inaccuracies in perfectly matching observed traffic counts, points are generally centered along the ideal diagonal and follow its trend. Inaccuracies are strongest in larger values and spread increases. Overall patterns are modeled correctly and show the models ability to forecast on long-term spans.

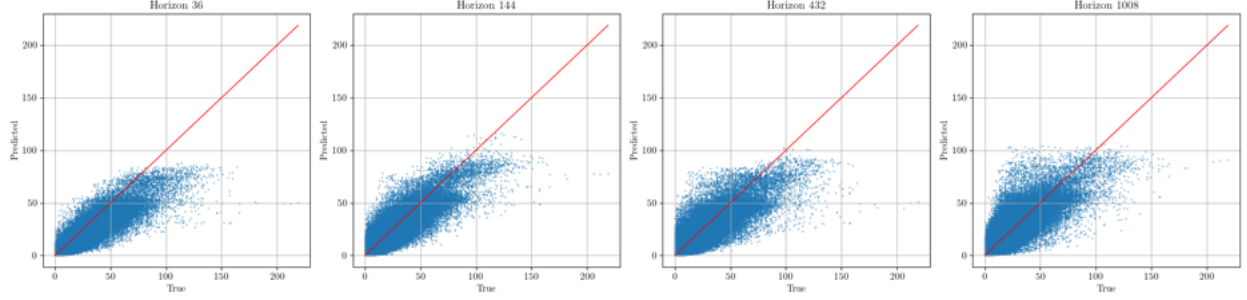


Figure 29: Scatterplots for True and Predicted Values per Horizon

The model’s stability in predicting long-term traffic can further be seen in Figure 30. All baselines show instability in future predictions, with MAE showing high variability and strong peaks. These approaches fail to generalize on longer time spans, preventing them from producing consistent forecasts. The MAE is often multiple times higher than the model’s and peaks increase sharply if unusual patterns occur. In contrast, the model maintains consistently low and stable error profiles. The graph shows no error accumulation or significant worsening on longer horizons. This is emphasized as graphs for the 6h horizon (Fig. 30a) are nearly identical to the 7 days horizon (Fig. 30b). Spatio-temporal patterns are learned consistently and largely uninfluenced by the forecasting length, showing the great detail at which short-term patterns were integrated into long-term trend learning.

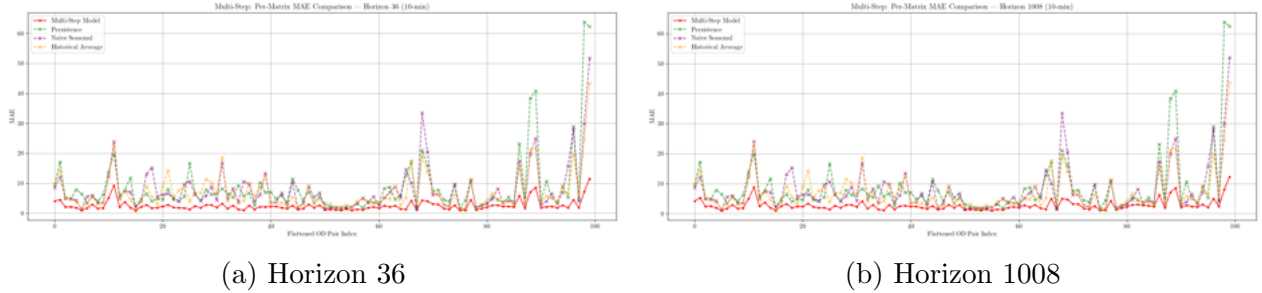


Figure 30: MAE Comparison for Model and Baselines for Short- and Long-Term Predictions

The low error scores, high cosine similarities, along with stable accuracy in short- and long-term traffic prediction, confirm the applicability of this approach.

4.5 Attention Distribution

To better explain the prediction mechanism, attention weights were directly extracted from the model and analyzed to allow interpretations of which spatial and temporal patterns were considered most relevant. These weights were compared against structural graph characteristics of the traffic network, distance between nodes, and OD flows. Spatial attention was additionally put in relation to prediction errors to highlight areas where the model focused on irrelevant areas. Temporal weights from the LSTM and short-term branches were examined to identify weighting patterns within 24-hour windows. To visualize temporal attention shifts in the model, temporal attention patterns were compared to time windows with different error magnitudes. To get further insights into the weighing process, Spearman correlation was computed between temporal attention, different structural features, and OD traffic signal characteristics.

4.5.1 Spatial Attention

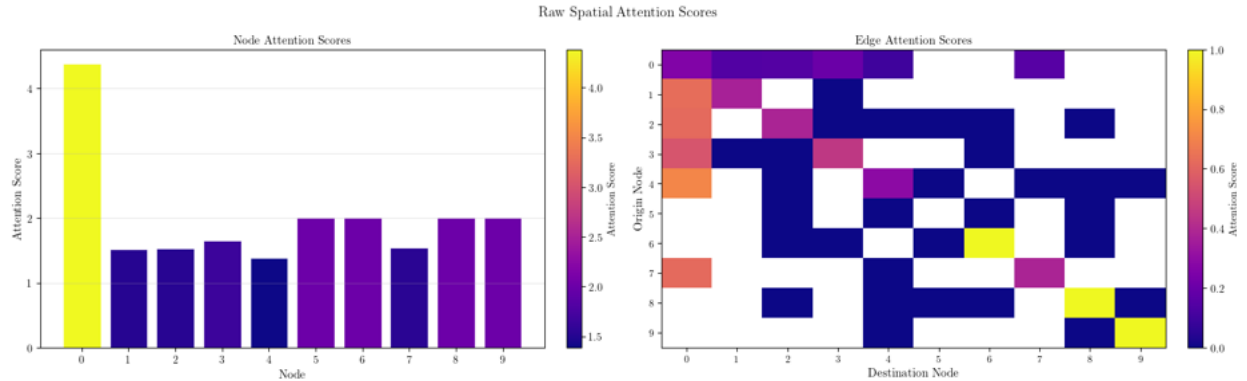


Figure 31: Global Attention Scores for Nodes and Edges

Spatial attention was assigned to each directed graph edge in the GAT to weigh spatial importance and identify patterns through space, determining how much information between neighboring nodes is considered in updating hidden states. To get an overview of this process and understand nodes and edges considered important by the model, spatial weights were aggregated over all time steps. Both nodes and edges were distinctly differently weighted and no clear patterns are visible (Fig. 31). Attention is largely centered around the diagonal and traffic coming or going towards node 0 (zone 237). Other edges were considered far less relevant with attention scores close to 0.

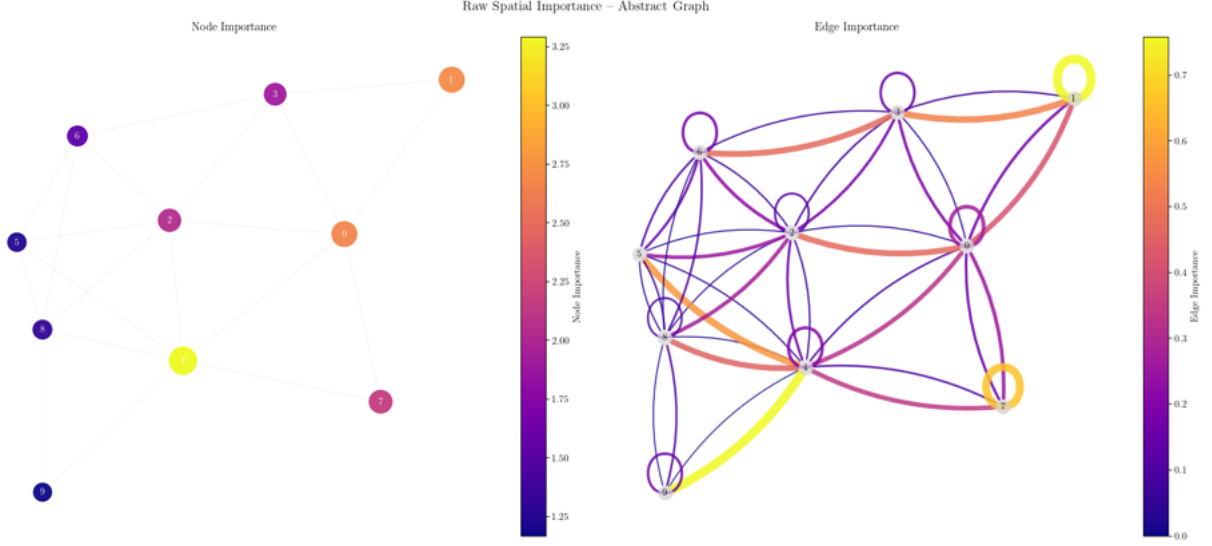


Figure 32: Spatial Attention Distribution on an Abstract Graph

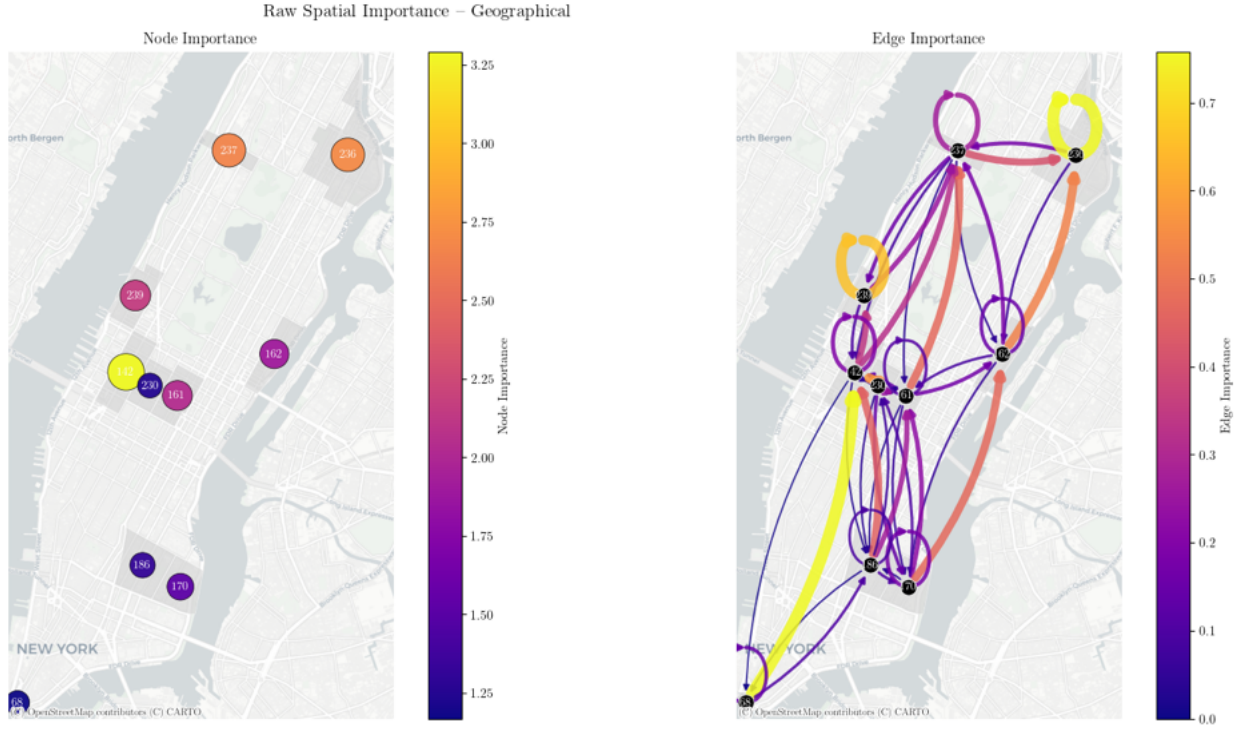


Figure 33: Spatial Attention Distribution on the Graph Projected onto a Map of Manhattan

Figures 32 and 33 show learned spatial attention on the graph in abstract form and projected onto a map of Manhattan. High attention was given to nodes 4, 1, and 0 (zones 142, 236 & 237), whereas nodes 8, 5, and 9 (zones 186, 230 & 68) were considered less relevant. No clear patterns are visible to why these nodes received stronger attention. While high connectivity

leads to generally larger attention strong differences remain. Outer nodes also show strongly different attention weights even if degree and neighborhood distances are similar. The location within the city also seemingly played a limited role. A general trend towards assigning larger importance to nodes towards the North and one central node exist, but the strongest weighted edge is on the opposite side of the graph.

Edge weights received equally mixed weights without clear patterns. While edges to important nodes were assigned larger weights, these weightings were not bi-directional and outgoing traffic was considered equally important at all nodes. Self-connections were equally viewed differently in regards to their importance. Although some general flow towards nodes 4 (142) and 1 (236) seems to have been prioritized, a clear explanation is not possible.

A statistical analysis of different graph characteristics and traffic flow further shows the unclear attention mechanism in the GAT. No significant correlations were found regarding weights and graph topology (Table 10).

Table 10: Spearman Correlation Between Structural Metrics and Total Attention

Metric	Spearman ρ	p -value
Total Degree	+0.291	0.415
Average Neighbor Distance	+0.212	0.556
Betweenness Centrality	+0.399	0.254
Closeness Centrality	+0.265	0.460
Eigenvector Centrality	+0.091	0.803
Average Inflow	−0.152	0.676
Average Outflow	−0.297	0.405
No correlations significant at $p < 0.05$ ($n = 10$ nodes).		

4.5.2 Temporal Attention

Temporal attention was calculated from the outputs of the long- and short-term branches of the model. Each produced a normalized attention vector, indicating the spatial weights of time steps considered important in making predictions. Analyzing the patterns produced in temporal learning reveals the different scales on which the model learned and which patterns were drawn from to make predictions of future traffic.

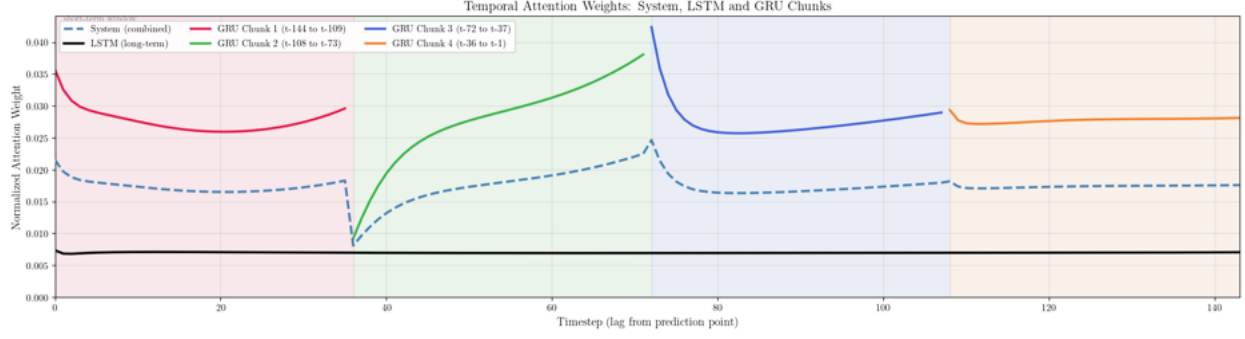


Figure 34: Branch- and System-wide Temporal Attention Distribution

Figure 34 visualizes how the long-term branch and the 6-hour short-term chunks differ in how they assign weights to individual time steps, showing their distinct roles in pattern learning clear. The GRU branches display distinct profiles, whereas the LSTM curve stays mostly flat. The short-term modules adjust their weights to capture recurring but highly variable patterns on short time spans, assigning different importance to specific time steps as needed. The LSTM, in contrast, spreads its attention more evenly across the full window, acting as a stabilizing long-term component. The combined global attention curve reflects how both mechanisms interact. It keeps the characteristic shape of the GRU profiles but is pulled toward a more uniform weighting by the LSTM, merging short-term fluctuations with broader long-term trends.

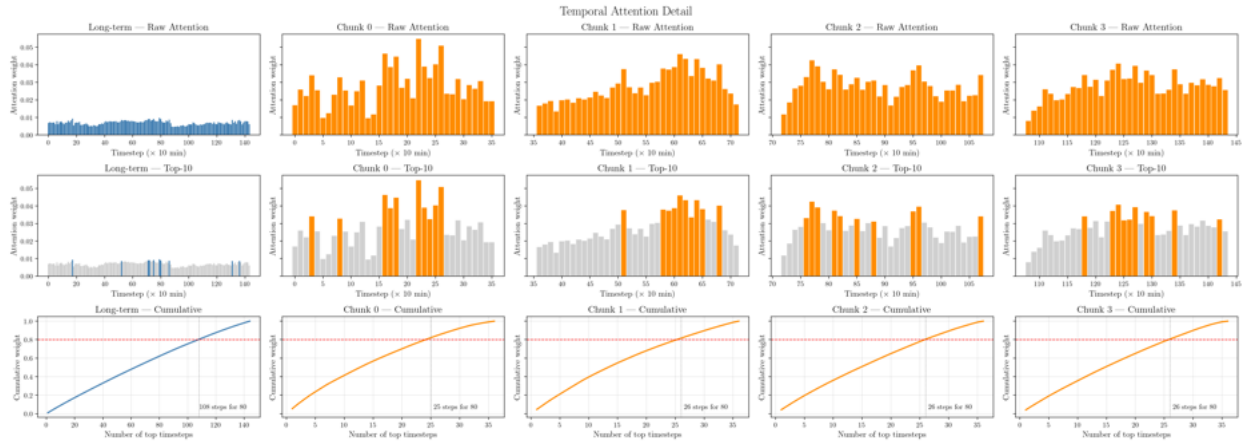


Figure 35: Detailed View of Assigned Temporal Weights in Different Branches

This observation is further visualized in Figure 35. The LSTM spread information across the entire window at a similar degree. Long-term patterns are encoded and weights adapted to learn traffic over 24-hour windows. Each GRU adapted its weights to match specific patterns within their assigned time frame. Important steps were identified and given greater attention

to adopt to more rapid change in traffic. High attention time steps are additionally more concentrated at specific points, showing the branches’ capability to identify time steps of greater importance to learn and predict more varying traffic behavior instead of relying on historic data.

To understand the variables influencing temporal attention weighing, Spearman’s ρ was calculated between weights and a series of data characteristic and prediction errors (Table 11). Strong correlations in traffic variance (0.613), autocorrelation (0.766), and traffic magnitude (0.547) show the LSTM tracking persistent traffic patterns. It predicts strongly auto-correlated patterns as they occur on a daily basis. The negative correlation with the prediction error (-0.256) shows positive influence of the LSTM on general predictions as learned long-term context is beneficial for future predictions. Overall small correlations for Chunk 0 covering nighttime and early morning hours, reflects the low amount of traffic and thus information available for pattern learning. As traffic volume rapidly rises during the morning until noon and remains mostly regular throughout the year, Chunk 1 shows near-perfect correlations with autocorrelation (-1) in the data. This is further reflected by the results for traffic magnitude (0.916) and seasonal phase (-1). As a result, the branches impact on predicting future traffic is extremely high with a negative correlation to prediction error of -0.989 and accurate attention assignment for these hours leads to better predictions. Similarly, evening traffic correctly captured by Chunk 3 has a similar effect and correlations match those of Chunk 0. The afternoon traffic is less relevant as shown by the values for Chunk 2.

Table 11: Spearman Correlation: Attention Branch vs. Signal

Branch	Traffic Variance	Pred. Error	Autocorrelation	Traffic Magnitude	Seasonal Phase
System	0.133	-0.121	-0.029	0.163	-0.424***
LSTM	0.613***	-0.256**	0.766***	0.547***	0.171*
Chunk 0	0.212	-0.345*	0.349*	0.181	-0.349*
Chunk 1	0.839***	-0.989***	-1.000***	0.916***	-1.000***
Chunk 2	0.250	-0.250	0.457**	0.250	0.250
Chunk 3	0.804***	0.787***	0.067	0.789***	-0.812***

* $p < 0.05$ ** $p < 0.01$ *** $p < 0.001$

The model’s reaction to prediction errors is shown in Figure 36. The temporal attention weights were systematically shifted to minimize prediction errors. When errors were low, the model relied mainly on recent information to predict future traffic. As predictions grew less accurate, the model shifted focus to data from earlier time steps. It mixed information between long- and short-term branches on demand to achieve better predictions and dynamically shift attention to the temporal context most relevant for reducing uncertainty. Medium errors show a broader spread of attention throughout the entire daily traffic, whereas very ac-

curate or inaccurate predictions lead to accessing weights concentrated during specific times. Stable patterns were confirmed or unusual patterns compensated.

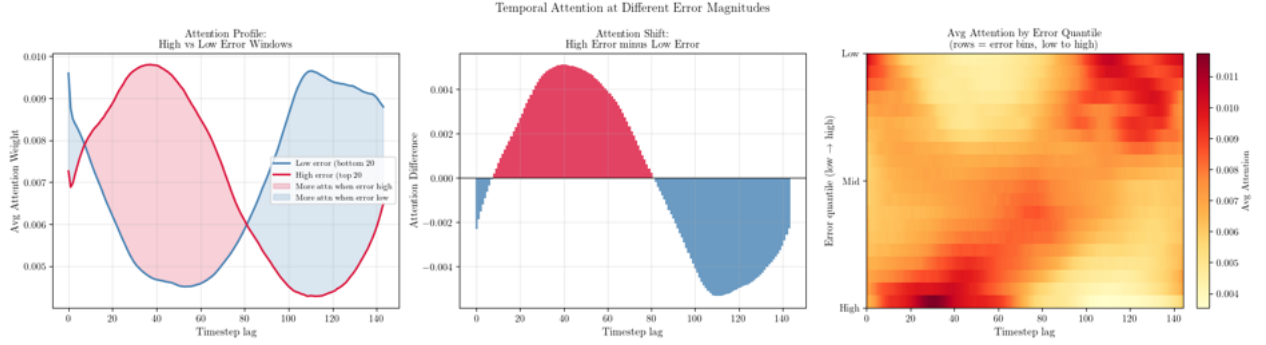


Figure 36: Temporal attention behavior conditioned on prediction error. Left: average attention profiles for low-error (bottom 20%) and high-error (top 20%) windows. Center: difference in attention weights (high minus low error). Right: average attention heatmap across 20 error quantile bins (rows ordered low to high error).

5 Discussion

5.1 Discussion of the Methods

Model Architecture

The results prove the model architecture useful for predicting traffic on a graph structure. In all tests, metrics indicate usefulness in application on real-world data. Very high cosine similarities and strong matching of ground truth pattern show the ability of the GAT to capture spatial relationships in the data. Based on minimal data, the model learned to successfully reproduce and predict relations of nodes and traffic. Experiments further show that the architecture does not simply memorize spatio-temporal correlations, but consistently identifies relationships through space and time. The use of attention mechanisms allowed the model to weigh influential relations on the graph as well as in the dynamic changes over time. Consistent results across different time spans show high levels of generalization and identification of importance that is not limited to local observations in data. Spatial patterns were kept across comparatively long time frames and could be reproduced days ahead. The learned representations were successfully encoding important and consistent structures of the graph. Further, the use of multiple GRUs was shown to be reasonable for short-term traffic modeling. Comparisons of predictions and true data across short windows of time show accurate prediction in rapid changes on smaller scales. The model was sensitive to rapid changes on sub-day scales and adapted appropriately. Splitting daily patterns among the different units helped to avoid overfitting on rush hours or losing detail during hours of lower traffic. The additional use of an LSTM to model longer trends proved reasonable in relation to long-term forecasting. Training on daily patterns helped achieving improved results on longer horizons and encode recurring patterns in traffic. Most importantly, the use of temporal attention to weigh importance of patterns in time helped improving long-term forecasting. Notable patterns could be kept and reproduced and inaccuracies from rapid changes largely mitigated. The general multi-branch architecture helped to separate daily patterns from finer grained local changes and neither were noticeably overrepresented. Analysis of the assigned weights show a fusion of different temporal scales in an accurate model that adapts dynamically to necessary information even if traffic shows unusual patterns.

Minor improvements can be made to the decoder or loss calculation and backpropagation. The model showed systemic bias towards underpredicting traffic, likely stemming from issues with mapping encodings to traffic volumes or an unfit loss function. Better optimization could further be achieved by penalizing underpredictions more strongly, longer training, or

additional data.

Data Pre-Processing

While the data pre-processing was useful to test the model on predicting traffic, the practical applicability is limited. Reducing the graph to ten nodes helped reduce computation times and complexity for model training but lost potential information at the same time. The taxi zones were clustered to large, elongated regions, not fully reflect the spatial structure of the city. Fine-grained patterns were smoothed and local variability was lost. Clustering based on overall traffic further reduced complexity and heterogeneity of traffic patterns. As a result, the model learned and predicted on a simplified graph that only approximates reality.

This restriction makes it difficult to address generalizability of the approach. It remains unclear how well the model would transfer to more complex networks with significantly larger numbers of zones. Further, aggregation simplified and often changed adjacency. Nodes that are not adjacent in reality became adjacent through assigned clusters. Similarly, calculating the distance matrix from static averages does not fully reflect reality. A more dynamic approach might further improve predictions by introducing changing patterns in distances driven.

Despite these limitations, the data was sufficient for experiments. The tests provided insights into the model’s ability to learn patterns and forecast traffic.

5.2 Discussion of the Results

Model Accuracy

The evaluation results demonstrate the model’s ability to predict traffic with high accuracy, both on short- and long-term scales. Error metrics are low throughout all horizons, with consistently low MAE and RMSE. The model did not struggle with forecasting into longer temporal distances, but managed to capture immediate as well as structural patterns in traffic. The consistency in predicting traffic in different and day-long horizons suggests strong capabilities in generalizing encoded spatio-temporal patterns beyond training data. High results for cosine similarities further emphasize this. For all prediction modes, the model was able to reproduce spatial patterns with great detail. Even when volumes differed, predictions reflected traffic flow directions highly accurately.

Long-term accuracy appeared to be surprisingly high and robust. Even at horizons of several

days, the model managed to reproduce traffic patterns and volumes. No error accumulation was visible and flows were estimate accurately. Results being similar through different time frames further emphasize the architecture’s suitability for the task.

Next-step predictions showed strong issues with underprediction. The results were universally biased and showed lower values in proportion to absolute traffic. While this did not impact long-term predictions or the learning of spatial and temporal patterns, it highlights reasons for adjusting the model to overcome this. Though immediate corrections were possible post-hoc, internal changes seem reasonable. As a simple linear regression on very small amounts of data was enough to overcome this bias, the issue does not appear to be fatal for assessment of the model’s predictive performance. Using a residual-based linear regression to correct proved to be useful. Even small amounts of calibration data helped reducing absolute errors between true and predicted data dramatically. The predictions aligned much more closely with ground-truth. This further showed the model accurately learned patterns and only failed to adjust its scale.

The improvements achieved by such a minimal correction highlight the general usefulness of the architecture. Fundamental information was extracted correctly and bias appeared as a minor side effect.

Long-Term Results

Most notable in experiments were long-term forecasting results. The model was found to be capable to predict future traffic across longer periods with very high accuracy. Extending temporal horizons showed only minimal degradation in results. Even when predicting several days into the future, errors remained small. Consistently high cosine similarities were observed through all horizons, showing the model effectively learned to generalize patterns across time. Model predictions were not overly dependent on short-term traffic but learned in a wider scope. The combination of an LSTM with spatial attention proved especially effective in maintaining accuracy across multiple days of forecasting on unseen data.

A minor drawback is the implementation of prediction on single target matrices. While this is likely computationally less expensive, predicting sequences could provide more realistically applicable results and full spatio-temporal patterns from single points.

Practical Importance

The proposed model is well-suited to address urban traffic problems. The utilization of the two-branch architecture allowed to capture traffic at different scales within the same

framework. By separating short-term and long-term learning, the model avoids trade-offs between accuracy across different scales. It successfully generalizes across time, allowing to capture both rapid changes and structural trends. This is essential for modern mobility management as it gives decision-makers access to forecasts that remain reliable across real-time and strategic future horizons. Short-term predictions support immediate interventions such as adaptive signal control, routing adjustments, or incident response, while long-term forecasts provide a stable foundation for evaluating infrastructure projects, mobility policies, and expected shifts in demand and use. A model capable of delivering consistent performance across these temporal ranges therefore offers practical value beyond conventional approaches, which typically specialize in only one horizon and fail to transfer knowledge across scales. By integrating both perspectives within a single architecture, the proposed model contributes to more coherent, data-driven traffic planning and enables a more resilient and sustainable management of urban mobility systems.

In addition, the approach addresses a gap that is largely ignored in current research. While short-term traffic forecasting has been studied extensively, existing deep learning models are almost exclusively designed for immediate or near-future prediction horizons. The overwhelming majority of published work focuses on forecasting the next few minutes or, at most, the next hour, and architectures are optimized accordingly. As a result, there are effectively no established deep learning models that explicitly target long-term traffic prediction or attempt to maintain stability across extended horizons. The proposed model directly fills this gap. By integrating separate branches for short- and long-term learning, it introduces a structure that is different from existing architectures, which generally rely on a single temporal representation. This design allows the model to retain sensitivity across horizons. In this sense, the contribution addresses a missing capability in the field: a deep learning model that is explicitly built for long-term forecasting.

5.3 Future Work

In future work, focus should lie on addressing the bias towards underprediction. Training on more data or epochs, or modifying the loss function and decoder could help overcoming the issue. Additionally, experiments on larger graphs or a wider set of input data, for example, weather or calendrical information, as well as dynamic distances or routing considerations, could further improve forecasting. As large improvements seem to stem from the LSTM, investigation into the impact of multiple GRUs could lead to additional insights. Extending the model to not only predict traffic volumes but additional parameters, such as traffic speed,

accident occurrence, or a range of economical or social variables, could further help with strategic planning tasks. Benchmarking the model against other deep learning architectures and testing its scalability on larger urban networks would provide further insights into its performance and practical applicability. Finally, the deployment of the model in a real-world setting with live recording of traffic could prove its usefulness.

6 Conclusion

This thesis investigated whether a graph-based deep learning architecture can be used to successfully predict traffic at different time scales and longer horizons. By combining a Graph Attention Network with GRUs, an LSTM, and temporal attention, spatio-temporal dependencies were modeled and patterns reoccurring learned. By splitting short- and long-term branches, complex change could be reproduced and applied to real-world data based on TLC data for urban traffic in New York City.

In all experiments, the model was shown to be capable to consistently achieve accurate results. Errors were low throughout all tests, with MAE ranging from 2.53 to 2.77 and RMSE values between 4.31 and 4.76. Predictions remained accurate on different horizons, even for forecasting frames of up to one week. The very high cosine similarities, often above 0.9, further showed the learning of spatial dependencies through time and the possibility to reproduce them for future traffic prediction. The GAT was successfully utilized to model important spatial relationships found in the data.

The model managed to forecast traffic at high accuracy. Predictions were made six hours, one day, three days, and seven days into the future, retaining spatial patterns and traffic volumes. Combining the LSTM with an attention mechanism helped learning long term trends even when input data was processed at a short-term basis. Forecasting a week into the future was very successful, with MAE and RMSE remaining at 2.77 and 4.79. Cosine similarities showed consistency, too, as they only dropped to 0.88. These results strongly highlight the model’s robustness and capability to generalize across time. Short-term predictions were made at similarly high accuracies. With an initial MAE of 2.69 and RMSE of 4.90, the model showed its ability to capture and reproduce traffic. The GRUs utilized for short-term traffic learning proved effective and managed to model detailed patterns very close to observations. Compared to simple baseline models, predictions were improved by 32 to 59%. With mean cosine similarities of 0.91, short-term traffic was modeled at extremely high accuracy.

Even though the experiments showed strong forecasting capabilities, they also revealed systemic bias in short-term, single-step predictions. Predicted values were consistently lower than corresponding true data, correlating with magnitude. However, this issue was overcome by applying a simple residual-based linear regression trained on minimal data. Single-step results were drastically improved, with MAE and RMSE reduced by 17% and 27%, respectively. This indicates that the architecture managed to successfully capture patterns in the data but ultimately failed to reproduce correct absolute volumes. Importantly, long-term

results were not suffering from similar bias.

Reducing the initial graph from the original 67 taxi zones of Manhattan into ten aggregated and continuous clusters helped reducing the complexity of the graph problem. At the same time, this introduced limitations. Spatial details were lost at finer scales and the resulting graph does not necessarily represent real-world conditions. At the same time, it helped with more efficient training and clearer analysis of results. The very strong performance under these conditions show the architecture’s capability of forecasting traffic at long time spans and short-term patterns simultaneously.

Overall, this thesis shows that the proposed model, with a combination of a GAT, GRUs, an LSTMs, and temporal attention, can provide a deep learning framework to effectively model traffic on a graph structure. Essential spatial and temporal patterns and relations are captured and can be reproduced to predict traffic across long time spans. The low errors and high cosine similarities, along with stable accuracy in short- and long-term traffic prediction, confirm the applicability of this approach.

References

- Attoui, M., & Lahby, M. (2025). A Systematic Literature Review of Traffic Congestion Forecasting: From Machine Learning Techniques to Large Language Models. *Vehicles*, 7(4), 142. <https://doi.org/10.3390/vehicles7040142>
- Bielecki, P., & Hachaj, T. (2026). Current Challenges and Issues in Car Traffic Forecasting. *IEEE Access*, 14, 8078–8106. <https://doi.org/10.1109/ACCESS.2026.3653629>
- Borgato, S., Fermi, F., Chirico, F., & Bosetti, S. (2021, November). Study on costs and benefits of the sustainable urban mobility transition – d3: Final report. <https://www.eiturbanmobility.eu/wp-content/uploads/2021/10/Final-report-Long-version.pdf>
- Castro-Neto, M., Jeong, Y.-S., Jeong, M.-K., & Han, L. D. (2009). Online-SVR for short-term traffic flow prediction under typical and atypical traffic conditions. *Expert Systems with Applications*, 36(3), 6164–6173. <https://doi.org/10.1016/j.eswa.2008.07.069>
- Chandra, S. R., & Al-Deek, H. (2009). Predictions of Freeway Traffic Speeds and Volumes Using Vector Autoregressive Models. *Journal of Intelligent Transportation Systems*, 13(2), 53–72. <https://doi.org/10.1080/15472450902858368>
- Chen, J., Wang, X., & Xu, X. (2021). Gc-lstm: Graph convolution embedded lstm for dynamic link prediction. <https://arxiv.org/abs/1812.04206>
- Chen, Y., Huang, J., Xu, H., Guo, J., & Su, L. (2023). Road traffic flow prediction based on dynamic spatiotemporal graph attention network. *Scientific Reports*. <https://doi.org/10.1038/s41598-023-41932-6>
- Cho, K., Courville, A., & Bengio, Y. (2015). Describing Multimedia Content using Attention-based Encoder–Decoder Networks [arXiv:1507.01053 [cs]]. *IEEE Transactions on Multimedia*, 17(11), 1875–1886. <https://doi.org/10.1109/TMM.2015.2477044>
- Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling [Version Number: 1]. <https://doi.org/10.48550/ARXIV.1412.3555>
- Duan, Y., Zhang, Y., Wang, X., Xue, Y., Wang, Z., & Wu, D. (2026). Feature-enhanced iTransformer: A two-stage framework for high-accuracy long-horizon traffic flow forecasting (G. Jin, Ed.). *PLOS One*, 21(1), e0340389. <https://doi.org/10.1371/journal.pone.0340389>
- European Commission. (2020, December). Sustainable and smart mobility strategy – putting european transport on track for the future. <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A52020DC0789>
- European Commission. (2019). *The european green deal* (Communication No. COM(2019) 640 final). European Commission. Brussels. eur-lex.europa.eu

- European Commission. Directorate General for Mobility and Transport. (2024). *Transport in the European Union: Current trends and issues*. Publications Office. Retrieved January 10, 2026, from <https://data.europa.eu/doi/10.2832/131741>
- European Court of Auditors. (2020, June). Sustainable urban mobility in the eu: No substantial improvement is possible without member states' commitment. <https://op.europa.eu/webpub/eca/special-reports/urban-mobility-6-2020/en/>
- Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In Y. W. Teh & M. Titterton (Eds.), *Proceedings of the thirteenth international conference on artificial intelligence and statistics* (pp. 249–256, Vol. 9). PMLR. <https://proceedings.mlr.press/v9/glorot10a.html>
- Guo, S., Lin, Y., Feng, N., Song, C., & Wan, H. (2019). Attention based spatial-temporal graph convolutional networks for traffic flow forecasting. *Proceedings of the Aaai Conference on Artificial Intelligence*. <https://doi.org/10.1609/aaai.v33i01.3301922>
- Hamed, M. M., Al-Masaeid, H. R., & Said, Z. M. B. (1995). Short-Term Prediction of Traffic Volume in Urban Arterials. *Journal of Transportation Engineering*, 121(3), 249–254. [https://doi.org/10.1061/\(ASCE\)0733-947X\(1995\)121:3\(249\)](https://doi.org/10.1061/(ASCE)0733-947X(1995)121:3(249))
- Han, S., Sun, Q.-W., Zhao, Q., Han, R., & Chen, Y. (2022). Traffic forecasting based on integration of adaptive subgraph reformulation and spatio-temporal deep learning model. *Electronics*. <https://doi.org/10.3390/electronics11060861>
- Homem De Gouveia, P., & Wojewódzka-Król, K. (2025, August). Opinion of the european economic and social committee - planning sustainable urban mobility in the eu. https://eur-lex.europa.eu/legal-content/EN/TXT/PDF/?uri=OJ%3AC_202504211&utm_source=copilot.com
- Jiang, W., & Luo, J. (2022). Graph neural network for traffic forecasting: A survey. *Expert Systems with Applications*, 207, 117921. <https://doi.org/10.1016/j.eswa.2022.117921>
- Jiang, W., Mu, J., Han, H., Zhang, Y., & Huang, S. (2024). Federated learning-based mobile traffic prediction in satellite-terrestrial integrated networks. *Software Practice and Experience*. <https://doi.org/10.1002/spe.3386>
- Jiang, X., & Adeli, H. (2005). Dynamic Wavelet Neural Network Model for Traffic Flow Forecasting. *Journal of Transportation Engineering*, 131(10), 771–779. [https://doi.org/10.1061/\(ASCE\)0733-947X\(2005\)131:10\(771\)](https://doi.org/10.1061/(ASCE)0733-947X(2005)131:10(771))
- Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization [Version Number: 9]. <https://doi.org/10.48550/ARXIV.1412.6980>
- Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. <https://arxiv.org/abs/1609.02907>

- Kong, X., Xing, W., Wei, X., Bao, P., Zhang, J., & Lu, W. (2020). Stgat: Spatial-temporal graph attention networks for traffic flow forecasting. *IEEE Access*. <https://doi.org/10.1109/access.2020.3011186>
- Li, M., & Zhu, Z. (2021). Spatial-temporal fusion graph neural networks for traffic flow forecasting. *Proceedings of the AAAI Conference on Artificial Intelligence*. <https://doi.org/10.1609/aaai.v35i5.16542>
- Li, Z., Wei, S., Wang, H., & Wang, C. (2024). Addgcn: A novel approach with down-sampling dynamic graph convolution and multi-head attention for traffic flow forecasting. *Applied Sciences*. <https://doi.org/10.3390/app14104130>
- Lippi, M., Bertini, M., & Frasconi, P. (2013). Short-Term Traffic Flow Forecasting: An Experimental Comparison of Time-Series Analysis and Supervised Learning. *IEEE Transactions on Intelligent Transportation Systems*, 14(2), 871–882. <https://doi.org/10.1109/TITS.2013.2247040>
- Liu, K., Zhu, Y., Wang, X., Ji, H., & Huang, C. (2024). Spatio-temporal graph attention convolution network for traffic flow forecasting. *Transportation Research Record Journal of the Transportation Research Board*. <https://doi.org/10.1177/03611981231225208>
- Lu, H., Huang, D., Song, Y., Jiang, D., Zhou, T., & Qin, J. (2020). St-trafficnet: A spatial-temporal deep learning network for traffic forecasting. *Electronics*, 9(9). <https://doi.org/10.3390/electronics9091474>
- Lv, M., Hong, Z., Chen, L., Chen, T., Zhu, T., & Ji, S. (2021). Temporal Multi-Graph Convolutional Network for Traffic Flow Prediction. *IEEE Transactions on Intelligent Transportation Systems*, 22(6), 3337–3348. <https://doi.org/10.1109/TITS.2020.2983763>
- Min, W., & Wynter, L. (2011). Real-time road traffic prediction with spatio-temporal correlations. *Transportation Research Part C: Emerging Technologies*, 19(4), 606–616. <https://doi.org/10.1016/j.trc.2010.10.002>
- Okutani, I., & Stephanedes, Y. J. (1984). Dynamic prediction of traffic volume through Kalman filtering theory. *Transportation Research Part B: Methodological*, 18(1), 1–11. [https://doi.org/10.1016/0191-2615\(84\)90002-X](https://doi.org/10.1016/0191-2615(84)90002-X)
- Park, C., Lee, C., Bahng, H., Tae, Y., Jin, S., Kim, K.-H., Ko, S., & Choo, J. (2020). St-grat: A novel spatio-temporal graph attention networks for accurately forecasting dynamically changing road speed. *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, 1215–1224. <https://doi.org/10.1145/3340531.3411940>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). Pytorch:

- An imperative style, high-performance deep learning library. <https://arxiv.org/abs/1912.01703>
- Sak, H., Senior, A., & Beaufays, F. (2014). Long Short-Term Memory Based Recurrent Neural Network Architectures for Large Vocabulary Speech Recognition [Version Number: 1]. <https://doi.org/10.48550/ARXIV.1402.1128>
- Salinas, D., Flunkert, V., Gasthaus, J., & Januschowski, T. (2020). Deepar: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3), 1181–1191. <https://doi.org/https://doi.org/10.1016/j.ijforecast.2019.07.001>
- Shen, Y., Wang, L., Zeng, Y., Gou, Z., Wang, C., & Yu, Z. (2025). Long-Term Traffic Flow Prediction for Highways Based on STLLformer Model. *Sustainability*, 17(22), 10078. <https://doi.org/10.3390/su172210078>
- Stathopoulos, A., & Karlaftis, M. G. (2003). A multivariate state space approach for urban traffic flow modeling and prediction. *Transportation Research Part C: Emerging Technologies*, 11(2), 121–135. [https://doi.org/10.1016/S0968-090X\(03\)00004-4](https://doi.org/10.1016/S0968-090X(03)00004-4)
- Sun, H., Tang, X., Lu, J., & Liu, F. (2023). Spatio-temporal graph neural network for traffic prediction based on adaptive neighborhood selection. *Transportation Research Record Journal of the Transportation Research Board*. <https://doi.org/10.1177/03611981231198851>
- Tang, J., & Zeng, J. (2021). Spatiotemporal gated graph attention network for urban traffic flow prediction based on license plate recognition data. *Computer-Aided Civil and Infrastructure Engineering*. <https://doi.org/10.1111/mice.12688>
- TLC. (2026). Tlc trip record data. <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- United Nations. (2015). *Sustainable development goals. transforming our world: The 2030 agenda for sustainable development*. <https://sustainabledevelopment.un.org/topics>
- United Nations. (2026). *Intelligent transportation systems and smart mobility in support of the un sustainable development goals*. <https://www.itu.int/en/ITU-T/ITS/Pages/SDG.aspx>
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2017). Graph Attention Networks [Version Number: 3]. <https://doi.org/10.48550/ARXIV.1710.10903>
- Vlahogianni, E. I., Karlaftis, M. G., & Golias, J. C. (2005). Optimized and meta-optimized neural networks for short-term traffic flow prediction: A genetic approach. *Transportation Research Part C: Emerging Technologies*, 13(3), 211–234. <https://doi.org/10.1016/j.trc.2005.04.007>

- Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Yu, P. S. (2021). A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*. <https://doi.org/10.1109/tnnls.2020.2978386>
- Yin, D., Xue, H., Prabowo, A., Ao, S., & Salim, F. (2024). XXLTraffic: Expanding and Extremely Long Traffic forecasting beyond test adaptation [Version Number: 3]. <https://doi.org/10.48550/ARXIV.2406.12693>
- Yu, B., Yin, H., & Zhu, Z. (2017). Spatio-temporal graph convolutional neural network: A deep learning framework for traffic forecasting. *CoRR*, *abs/1709.04875*. <http://arxiv.org/abs/1709.04875>
- Zhang, H., Chen, L., Cao, J., Zhang, X., & Kan, S. (2021). A combined traffic flow forecasting model based on graph convolutional network and attention mechanism. *International Journal of Modern Physics C*. <https://doi.org/10.1142/s0129183121501588>
- Zhang, H., Wang, H., Zhang, X., & Gong, L. (2023). Research on traffic flow forecasting based on dynamic spatial-temporal transformer. *Transportation Research Record Journal of the Transportation Research Board*. <https://doi.org/10.1177/03611981231205880>
- Zhao, L., Song, Y., Zhang, C., Liu, Y., Wang, P., Lin, T., Deng, M., & Li, H. (2020). T-gcn: A temporal graph convolutional network for traffic prediction. *IEEE Transactions on Intelligent Transportation Systems*. <https://doi.org/10.1109/tits.2019.2935152>
- Zheng, C., Fan, X., Wang, C., & Qi, J. (2020). Gman: A graph multi-attention network for traffic prediction. *Proceedings of the Aaai Conference on Artificial Intelligence*. <https://doi.org/10.1609/aaai.v34i01.5477>
- Zhu, J., Tao, C., Deng, H., Zhao, L., Wang, P., Lin, T., & Li, H. (2020). AST-GCN: attribute-augmented spatiotemporal graph convolutional network for traffic forecasting. *CoRR*, *abs/2011.11004*. <https://arxiv.org/abs/2011.11004>
- Zhu, T., Boada, M. J. L., & Boada, B. L. (2024). Adaptive graph attention and long short-term memory-based networks for traffic prediction. *Mathematics*, *12*(2). <https://doi.org/10.3390/math12020255>

Appendix A Model

```

1 class FusedODModel(nn.Module):
2     def __init__(self, N, in_features, hidden_size=64, W_long=144,
3         chunk_size_short=9, num_chunks_short=4, num_prediction_horizons=1,
4         adj_matrix_tensor: torch.Tensor = None, dist_matrix_tensor: torch.
5         Tensor = None):
6         super().__init__()
7         self.N = N
8         self.W_long = W_long # Total input window length for the long-term
9         branch
10
11         self.chunk_size_short = chunk_size_short # Length of each chunk
12         for the short-term GRU branch
13         self.num_chunks_short = num_chunks_short # Number of chunks for
14         the short-term GRU branch (= number of GRUs)
15         self.W_short = self.chunk_size_short * self.num_chunks_short #
16         Total window size for the short-term branch
17         self.num_prediction_horizons = num_prediction_horizons # Now a
18         parameter
19
20         # GCN/GAT layer shared by both long-term and short-term branches
21         self.gcn = GraphAttentionLayer(in_features, hidden_size)
22
23         # Long-term branch: single LSTM with a single Temporal Attention
24         self.lstm_long = nn.LSTM(hidden_size, hidden_size, batch_first=
25         True)
26         self.temporal_attention_long = TemporalAttention(hidden_size)
27         self.register_buffer("feat_buffer_long", torch.empty(1, self.
28         W_long, hidden_size))
29
30         # Short-term branch: multiple GRUs, each with its own Temporal
31         Attention
32         self.grus_short = nn.ModuleList([nn.GRU(hidden_size, hidden_size,
33         batch_first=True) for _ in range(self.num_chunks_short)])
34         self.temporal_attentions_short = nn.ModuleList([TemporalAttention(
35         hidden_size) for _ in range(self.num_chunks_short)])
36         self.register_buffer("feat_buffer_short", torch.empty(1, self.
37         chunk_size_short, hidden_size))
38
39         # Fusion layer inputs: context vector from long-term LSTM (
40         hidden_size)
41         # + concatenated context vectors from short-term GRUs (
42         num_chunks_short * hidden_size)
43         fusion_input_dim = hidden_size + (hidden_size * self.

```

```

num_chunks_short)
27     self.linear1 = nn.Linear(fusion_input_dim, hidden_size * 2)
28     self.relu1 = nn.ReLU()
29     # Adjusted output dimension of linear2 to accommodate
num_prediction_horizons
30     self.linear2 = nn.Linear(hidden_size * 2, self.
num_prediction_horizons * N * N)
31     self.conv2d = nn.Conv2d(in_channels=1, out_channels=1, kernel_size
=3, padding=1)
32     self.softplus = nn.Softplus()
33
34     # Register adjacency and distance matrices as buffers
35     if adj_matrix_tensor is not None:
36         self.register_buffer('adj_matrix_tensor', adj_matrix_tensor.
float())
37
38     if dist_matrix_tensor is not None:
39         self.register_buffer('dist_matrix_tensor', dist_matrix_tensor.
float())
40
41     def forward(self, seg: torch.Tensor):
42         B, total_W, _, _ = seg.size()
43
44         # Ensure that the total_W matches W_long as the main input to the
model
45         if total_W != self.W_long:
46             raise ValueError(f"Input segment length {total_W} does not
match expected W_long {self.W_long}")
47
48         # --- Long-term branch processing ---
49         # Prepare buffer for GCN outputs over the entire W_long sequence
50         long_term_gcn_features = self.feats_buffer_long.expand(B, -1, -1).
clone() # (B, W_long, hidden_size)
51         for t in range(self.W_long):
52             # Apply GCN to each time step in the long-term input segment
53             H = self.gcn(seg[:, t, :, :], self.adj_matrix_tensor, self.
dist_matrix_tensor)
54             long_term_gcn_features[:, t] = H.mean(dim=1) # Aggregate node
features (mean over nodes)
55
56         # Pass GCN features through the long-term LSTM
57         lstm_output_long, _ = self.lstm_long(long_term_gcn_features) # (B,
W_long, hidden_size)
58         # Apply temporal attention to get a context vector for the long-

```

```

term branch
    context_vector_long = self.temporal_attention_long(
lstm_output_long) # (B, hidden_size)

    # --- Short-term branch processing ---
    short_term_context_vectors = []
    # The short-term branch operates on the most recent W_short steps
of the input.
    seg_short_input = seg[:, -self.W_short:, :, :]

    for i in range(self.num_chunks_short):
        start = i * self.chunk_size_short
        end = start + self.chunk_size_short
        # Extract the current short-term chunk from the '
seg_short_input'
        chunk_short = seg_short_input[:, start:end, :, :] # (B,
chunk_size_short, N, N)

        # Prepare buffer for GCN outputs for this short-term chunk
        short_term_gcn_features = self.feats_buffer_short.expand(B, -1,
-1).clone() # (B, chunk_size_short, hidden_size)
        for t in range(self.chunk_size_short):
            # Apply GCN to each time step in the short-term chunk
            H = self.gcn(chunk_short[:, t, :, :], self.
adj_matrix_tensor, self.dist_matrix_tensor)
            short_term_gcn_features[:, t] = H.mean(dim=1) # Aggregate
node features (mean over nodes)

        # Pass GCN features through the respective short-term GRU
        gru_output_short, _ = self.grus_short[i](
short_term_gcn_features) # (B, chunk_size_short, hidden_size)
        # Apply temporal attention to get a context vector for this
short-term GRU
        context_vector_short_i = self.temporal_attentions_short[i](
gru_output_short) # (B, hidden_size)
        short_term_context_vectors.append(context_vector_short_i)

    # --- Fusion ---
    # Concatenate the long-term context vector with all short-term
context vectors
    fused = torch.cat([context_vector_long] +
short_term_context_vectors, dim=1) # (B, fusion_input_dim)

    # Decoder operations with Conv2d and Softplus

```



```
90     out = self.relu1(self.linear1(fused))
91     out = self.linear2(out)
92     # Reshape for multi-horizon output: (B, num_prediction_horizons, N
, N)
93     out = out.view(B, self.num_prediction_horizons, self.N, self.N)
94
95     # Adapt Conv2d to process each predicted horizon independently
96     # Reshape to (B * num_prediction_horizons, 1, N, N) for Conv2d
97     conv_input = out.view(B * self.num_prediction_horizons, 1, self.N,
self.N)
98     conv_output = self.conv2d(conv_input)
99     # Reshape back to (B, num_prediction_horizons, N, N)
100     out = conv_output.view(B, self.num_prediction_horizons, self.N,
self.N)
101     out = self.softplus(out)
102
103     return out
```

Appendix B Graph Attention Network

```
1 class GraphAttentionLayer(nn.Module):
2     # Basic single-head Graph Attention Layer. (Velickovic et al., 2018;
https://arxiv.org/abs/1710.10903)
3     def __init__(self, in_features: int, out_features: int, dropout: float
= 0.6, alpha: float = 0.2):
4         super(GraphAttentionLayer, self).__init__()
5         self.dropout = dropout
6         self.in_features = in_features
7         self.out_features = out_features
8         self.alpha = alpha # Negative slope of LeakyReLU
9
10        self.W = nn.Parameter(torch.empty(size=(in_features, out_features)
))
11        # initial weights via Xavier Uniform weight assignment
12        nn.init.xavier_uniform_(self.W.data, gain=1.414)
13
14        # attention
15        self.a = nn.Parameter(torch.empty(size=(2*out_features, 1)))
16        nn.init.xavier_uniform_(self.a.data, gain=1.414)
17
18        self.leakyrelu = nn.LeakyReLU(self.alpha)
19
```

```
20     # Learnable scalar to weigh the influence of distance
21     self.distance_scaler = nn.Parameter(torch.ones(1))
22
23     def forward(self, h: torch.Tensor, adj: torch.Tensor, dist: torch.
Tensor):
24         h_prime = torch.einsum("bnf,fh->bnh", h, self.W)
25
26         h_prime_i_repeat = h_prime.unsqueeze(2).repeat(1, 1, h_prime.shape
[1], 1)
27         h_prime_j_repeat = h_prime.unsqueeze(1).repeat(1, h_prime.shape
[1], 1, 1)
28
29         e_input = torch.cat((h_prime_i_repeat, h_prime_j_repeat), dim=3)
30
31         e_feature_based = self.leakyrelu(torch.einsum("bnij,jk->bnik",
e_input, self.a)).squeeze(3)
32
33         # Expand distance matrix for batch and apply log transformation
34         dist_expanded = dist.unsqueeze(0).expand_as(e_feature_based)
35         # Use negative log distance, scaled by learnable parameter. Add
epsilon to prevent log(0).
36         distance_influence = -torch.log(dist_expanded + 1e-6) * self.
distance_scaler
37
38         # Combine feature-based attention with distance influence
39         combined_attention_scores = e_feature_based + distance_influence
40
41         zero_vec = -9e15 * torch.ones_like(combined_attention_scores)
42         adj_expanded = adj.unsqueeze(0).expand_as(
combined_attention_scores)
43         attention = torch.where(adj_expanded > 0,
combined_attention_scores, zero_vec)
44
45         attention = F.softmax(attention, dim=2)
46         attention = F.dropout(attention, self.dropout, training=self.
training)
47
48         h_prime_aggregated = torch.einsum("bij,bjh->bih", attention,
h_prime)
49
50         return F.elu(h_prime_aggregated)
```

Appendix C Temporal Attention

```

1
2 class TemporalAttention(nn.Module):
3     # Simple Temporal Attention Layer
4     def __init__(self, hidden_size):
5         super(TemporalAttention, self).__init__()
6         # Learnable weight vector for attention
7         self.attn_weights = nn.Parameter(torch.rand(hidden_size, 1))
8         # Query transformation
9         self.W_q = nn.Linear(hidden_size, hidden_size)
10        # Key transformation
11        self.W_k = nn.Linear(hidden_size, hidden_size)
12        # Attention mechanism vector
13        self.v = nn.Parameter(torch.rand(hidden_size))
14
15        def forward(self, encoder_outputs): #(B, seq_len, hidden_size)
16            # Energy: (B, seq_len, 1)
17            energy = torch.tanh(torch.matmul(encoder_outputs, self.
18            attn_weights))
19            # Attention weights: (B, seq_len)
20            attention_weights = F.softmax(energy.squeeze(2), dim=1)
21            # Context vector: (B, hidden_size)
22            context_vector = torch.einsum("bs,bsd->bd", attention_weights,
23            encoder_outputs)
24            return context_vector

```

Appendix D Decoder

```

1 # Fusion layer inputs: context vector from long-term LSTM
2 # + concatenated context vectors from short-term GRUs
3 fusion_input_dim = hidden_size + (hidden_size * self.num_chunks_short)
4 self.linear1 = nn.Linear(fusion_input_dim, hidden_size * 2)
5 self.relu1 = nn.ReLU()
6 self.linear2 = nn.Linear(hidden_size * 2,
7                             self.num_prediction_horizons * N * N)
8 self.conv2d = nn.Conv2d(in_channels=1, out_channels=1,
9                             kernel_size=3, padding=1)
10 self.softplus = nn.Softplus()

```

Appendix E NB NLL Loss

```

1 class NegativeBinomialNLLLoss(nn.Module):
2     def __init__(self, eps=1e-8):
3         super().__init__()
4         self.eps = eps
5
6     def forward(self, mu, target, r):
7         # mu: predicted mean (from model output)
8         # target: true counts
9         # r: dispersion parameter (learned or fixed)
10
11        # Ensure mu and r are positive
12        mu = mu + self.eps
13        r = r + self.eps
14
15        log_gamma_target_plus_r = torch.lgamma(target + r)
16        log_gamma_target_plus_1 = torch.lgamma(target + 1)
17        log_gamma_r = torch.lgamma(r)
18
19        term1 = log_gamma_target_plus_r - log_gamma_target_plus_1 -
20        log_gamma_r
21        term2 = r * torch.log(r / (r + mu))
22        term3 = target * torch.log(mu / (r + mu))
23
24        loss = -torch.sum(term1 + term2 + term3)
25        return loss

```

Appendix F Model Training

```

1 # builds batch windows for the FusedODModel
2 # returns long input segments and multi-step target segments
3 def build_windows_generator_multi_step_fused(series_OD: torch.Tensor,
4       W_long: int, prediction_horizons: list, batch_size: int):
5     if series_OD.is_cuda:
6         series_OD = series_OD.cpu()
7
8     T, N, _ = series_OD.shape
9
10    # Determine the maximum horizon needed to calculate the total required
11    # length of the series
12    max_horizon = max(prediction_horizons)

```

```

11
12     # The effective number of windows must account for both the input
13     window and the furthest prediction horizon
14     num_windows = max(0, T - W_long - max_horizon + 1)
15     num_batches = (num_windows + batch_size - 1) // batch_size # Ceiling
16     division
17
18     for i in range(num_batches):
19         start_window_idx = i * batch_size
20         end_window_idx = min(num_windows, start_window_idx + batch_size)
21
22         X_batch_long = []
23         Y_batch_multi_step = [] # This will hold targets for all horizons
24
25         for t in range(start_window_idx, end_window_idx):
26             # Long-term input window: [t, t + W_long - 1]
27             X_batch_long.append(series_OD[t : t + W_long])
28
29             # Collect targets for all prediction horizons for the current
30             't'
31             current_horizons_targets = []
32             for horizon in prediction_horizons:
33                 # Target is the OD matrix at t + W_long + horizon - 1
34                 # The -1 is because horizon 1 means 1 step *after* the
35                 input window, which is index 't+W_long'
36                 # So, for horizon 'h', the index is 't + W_long + h - 1'
37                 target_idx = t + W_long + horizon - 1
38                 current_horizons_targets.append(series_OD[target_idx])
39
40             Y_batch_multi_step.append(torch.stack(current_horizons_targets
41 , dim=0)) # Stacks N*N matrices into (num_horizons, N, N)
42
43         X_batch_long = torch.stack(X_batch_long, axis=0).float()
44         Y_batch_multi_step = torch.stack(Y_batch_multi_step, axis=0).float
45         () # (Batch_size, num_horizons, N, N)
46
47         yield X_batch_long, Y_batch_multi_step
48
49 # multi-step training
50 def train_model_generator_multi_step_fused(model, series_OD: torch.Tensor,
51     W_long: int, prediction_horizons: list, lr: float, epochs: int, bs:
52     int, ckpt_path: str, device: torch.device, patience: int = 10,
53     min_delta: float = 1e-4, use_poisson_loss: bool = False):
54     # Initialize dispersion parameter as a learnable parameter ONLY if not

```

```

using Poisson
46 r_param = None
47 if not use_poisson_loss:
48     r_param = nn.Parameter(torch.tensor(1.0, device=device))
49     optimizer = torch.optim.Adam(list(model.parameters()) + [r_param],
lr=lr)
50     loss_fn = NegativeBinomialNLLLoss() # Use custom Negative Binomial
NLL Loss
51 else:
52     optimizer = torch.optim.Adam(model.parameters(), lr=lr)
53     loss_fn = nn.PoissonNLLLoss(log_input=False)
54
55     scaler = GradScaler()
56     scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode
='min', factor=0.5, patience=5)
57
58     best_loss = float("inf")
59     losses = []
60
61     model.train()
62
63     for ep in range(1, epochs + 1):
64         total = 0.0
65         count = 0
66         # Use the multi-step window generator
67         data_generator = build_windows_generator_multi_step_fused(
series_OD, W_long, prediction_horizons, bs)
68
69         for xb_long, yb_multi_step in data_generator:
70             xb_long, yb_multi_step = xb_long.to(device), yb_multi_step.to(
device) # yb_multi_step is (B, num_horizons, N, N)
71             optimizer.zero_grad()
72             with autocast():
73                 pred_multi_step = model(xb_long) # pred_multi_step will be
(B, num_horizons, N, N)
74                 pred_multi_step = torch.clamp(pred_multi_step, min=1e-8) #
Ensure mu is positive
75
76                 if not use_poisson_loss:
77                     # Ensure r is positive using softplus
78                     r_positive = F.softplus(r_param) # Apply softplus to
ensure r is positive
79                     loss = loss_fn(pred_multi_step, yb_multi_step,
r_positive)

```

```

80         else:
81             loss = loss_fn(pred_multi_step, yb_multi_step)
82             scaler.scale(loss).backward()
83             scaler.step(optimizer)
84             scaler.update()
85             total += loss.item()
86             count += 1
87
88     avg = total / count if count > 0 else 0
89     losses.append(avg)
90
91     if not use_poisson_loss:
92         print(f"Ep {ep:02d} \u2014 Train NB NLL Loss: {avg:.6f},
93               r_param: {F.softplus(r_param).item():.4f}")
94     else:
95         print(f"Ep {ep:02d} \u2014 Train Poisson NLL Loss: {avg:.6f}")
96
97     scheduler.step(avg)
98
99     # Save the model state at the end of each epoch if it's the best
100    so far
101    if avg < best_loss:
102        best_loss = avg
103        save_dict = {'model_state_dict': model.state_dict()}
104        if not use_poisson_loss:
105            save_dict['r_param'] = r_param.item()
106        torch.save(save_dict, ckpt_path)
107        print(f"New best model saved with loss: {best_loss:.6f}")
108
109    # Save the model every 10 epochs
110    if ep % 10 == 0:
111        periodic_ckpt_path = ckpt_path.replace(".pth", f"_epoch_{ep}.
112        pth")
113        save_dict = {'model_state_dict': model.state_dict()}
114        if not use_poisson_loss:
115            save_dict['r_param'] = r_param.item()
116        torch.save(save_dict, periodic_ckpt_path)
117        print(f"Saved model checkpoint at epoch {ep} to {
118        periodic_ckpt_path}")
119
120    return losses

```

Appendix G Long-term Prediction

```

1 def predict_long_future_patterns_fused(model, initial_input_window_long:
  torch.Tensor, num_steps_to_predict: int, device: torch.device):
2     model.eval()
3     with torch.no_grad():
4         current_input_window_long = initial_input_window_long.clone().to(
  device) # (1, W_long, N, N)
5         all_multi_horizon_predictions = [] # Store each multi-horizon
  prediction output from each recursive step
6
7         W_long = model.W_long
8         # W_short is no longer directly used in this function's input
  handling but is an internal model param
9
10        for _ in range(num_steps_to_predict):
11            # Model outputs num_prediction_horizons steps: (1,
  num_prediction_horizons, N, N)
12            multi_horizon_prediction = model(current_input_window_long)
13            multi_horizon_prediction = torch.clamp(
  multi_horizon_prediction, min=1e-6) # Ensure non-negativity
14
15            # Store the entire multi-horizon prediction output of this
  step
16            all_multi_horizon_predictions.append(multi_horizon_prediction.
  cpu().squeeze(0)) # (num_prediction_horizons, N, N)
17
18            # For recursive update, use the prediction for the first
  horizon (next immediate step)
19            next_step_prediction = multi_horizon_prediction[:, 0, :, :].
  unsqueeze(1) # (1, 1, N, N)
20            current_input_window_long = torch.cat((
  current_input_window_long[:, 1:, :, :], next_step_prediction), dim=1)
21
22        return torch.stack(all_multi_horizon_predictions, dim=0) # (
  num_steps_to_predict, num_prediction_horizons, N, N)

```

Appendix H Model Evaluation

```

1 # error metrics
2 def compute_metrics(y_true, y_pred, eps=1e-6):
3     yt, yp = y_true.ravel(), y_pred.ravel()

```



```

4     mse = mean_squared_error(yt, yp)
5     rmse = np.sqrt(mse)
6     mae = mean_absolute_error(yt, yp)
7     mask = np.abs(yt) > eps
8     mape = np.mean(np.abs((yt[mask] - yp[mask]) / yt[mask])) * 100 if mask
9     .any() else np.nan
10    smape = np.mean(2 * np.abs(yt - yp) / (np.abs(yt) + np.abs(yp) + eps))
11    * 100
12    r2 = r2_score(yt, yp)
13    return {"MSE": mse, "RMSE": rmse, "MAE": mae, "MAPE(%)": mape, "SMAPE
14    (%)": smape, "R2": r2}
15
16 def eval_fused_model_multi_step(model, series_OD: torch.Tensor, W_long:
17 int, prediction_horizons: list, bs: int, device: torch.device):
18     model.eval()
19
20     num_prediction_horizons = len(prediction_horizons)
21
22     # Initialize lists to store predictions and true values for each
23     horizon
24     all_preds_by_horizon = [[] for _ in range(num_prediction_horizons)]
25     all_trues_by_horizon = [[] for _ in range(num_prediction_horizons)]
26
27     test_data_generator = build_windows_generator_multi_step_fused(
28     series_OD, W_long, prediction_horizons, bs)
29
30     # Calculate total number of windows for tqdm progress bar
31     T, N_dummy, _ = series_OD.shape
32     max_horizon = max(prediction_horizons)
33     num_windows_test = max(0, T - W_long - max_horizon + 1)
34     num_batches_test = (num_windows_test + bs - 1) // bs
35
36     try:
37         from tqdm.auto import tqdm
38         batch_iterator = tqdm(test_data_generator, total=num_batches_test,
39         desc="Evaluating Fused Multi-Step Model")
40     except ImportError:
41         batch_iterator = test_data_generator
42
43     with torch.no_grad():
44         for xb_long, yb_multi_step in batch_iterator:
45             xb_long = xb_long.to(device)
46             # yb_multi_step is already moved to device in the generator

```

```
41         pred_multi_step = model(xb_long) # (B, num_horizons, N, N)
42         pred_multi_step = torch.clamp(pred_multi_step, min=1e-6) #
Ensure non-negativity
43
44         # Move to CPU and convert to numpy
45         pred_np = pred_multi_step.cpu().numpy()
46         true_np = yb_multi_step.cpu().numpy()
47
48         for h_idx in range(num_prediction_horizons):
49             all_preds_by_horizon[h_idx].append(pred_np[:, h_idx, :,
:]))
50             all_trues_by_horizon[h_idx].append(true_np[:, h_idx, :,
:]))
51
52         # Concatenate results for each horizon
53         final_preds_by_horizon = [np.concatenate(preds_list, axis=0) for
preds_list in all_preds_by_horizon]
54         final_trues_by_horizon = [np.concatenate(trues_list, axis=0) for
trues_list in all_trues_by_horizon]
55
56         # Compute metrics for each horizon
57         metrics_by_horizon = {}
58         for h_idx, horizon in enumerate(prediction_horizons):
59             y_true_horizon = final_trues_by_horizon[h_idx]
60             y_pred_horizon = final_preds_by_horizon[h_idx]
61             metrics_by_horizon[f"Horizon_{horizon}steps"] = compute_metrics(
y_true_horizon, y_pred_horizon)
62
63         return final_trues_by_horizon, final_preds_by_horizon,
metrics_by_horizon
```

Appendix I Baseline Models

```

1
2 def generate_baseline_predictions_multi_step_from_train(
3     train_OD: torch.Tensor,
4     test_OD: torch.Tensor,
5     prediction_horizons: list,
6     seasonal_period: int):
7
8     train_np = train_OD.cpu().numpy()
9     test_np = test_OD.cpu().numpy()
10
11     T_train = train_np.shape[0]
12     T_test = test_np.shape[0]
13
14     y_true_by_horizon = []
15     persistence_preds_by_horizon = []
16     naive_seasonal_preds_by_horizon = []
17     hist_avg_preds_by_horizon = []
18
19     for horizon in prediction_horizons:
20         y_true_list, pers_list, seas_list, hist_list = [], [], [], []
21
22         # For each test index t, true target at t + horizon
23         max_t = T_test - horizon
24         for t in range(max_t):
25             true_idx = t + horizon
26             y_true_list.append(test_np[true_idx])
27
28             # Persistence: last training value
29             pers_list.append(train_np[-1])
30
31             # Seasonal: value from seasonal_period before end of train
32             seasonal_idx = T_train - seasonal_period
33             if seasonal_idx >= 0:
34                 seas_list.append(train_np[seasonal_idx])
35             else:
36                 seas_list.append(train_np[-1])
37
38             # Historical average over last few seasonal lags
39             hist_samples = []
40             for k in range(1, 5):
41                 idx = T_train - k * seasonal_period
42                 if idx >= 0:

```

```
43         hist_samples.append(train_np[idx])
44     if hist_samples:
45         hist_list.append(np.mean(np.stack(hist_samples, axis=0),
axis=0))
46     else:
47         hist_list.append(train_np[-1])
48
49     y_true_by_horizon.append(np.stack(y_true_list, axis=0))
50     persistence_preds_by_horizon.append(np.stack(pers_list, axis=0))
51     naive_seasonal_preds_by_horizon.append(np.stack(seas_list, axis=0)
)
52     hist_avg_preds_by_horizon.append(np.stack(hist_list, axis=0))
53
54     return (
55         y_true_by_horizon,
56         persistence_preds_by_horizon,
57         naive_seasonal_preds_by_horizon,
58         hist_avg_preds_by_horizon
59     )
```